

TARTU ÜLIKOOL
Arvutiteaduse instituut
Informaatika õppekava

Uku Renek Kronbergs
Turvalise paroolihalduri veebirakenduse
arendamine
Bakalaureusetöö (9 EAP)

Juhendaja:
Alo Peets, MSc

Tartu 2026

Turvalise paroolihalduri veebirakenduse arendamine

Lühikokkuvõte:

Käesolev bakalaureusetöö käsitleb turvalise paroolihalduri veebirakenduse kavandamist ja arendamist. Paroolid on endiselt kõige levinum autentimismeetod, kuid nõrk paroolihaldus on üks peamisi küberturvalisuse riske. Töö eesmärk on luua null-teadmise arhitektuuril põhinev paroolihaldur, kus kogu krüpteerimine ja dekrüpteerimine toimub kasutaja brauseris ning teenusepakkuja ei saa kunagi ligipääsu krüpteerimata andmetele. Töös analüüsitakse olemasolevaid paroolihaldureid (Bitwarden, 1Password, LastPass, KeePass), kirjeldatakse arendatud rakenduse arhitektuuri ja turvamudelit ning esitatakse tehniline realiseerimine. Rakendus kasutab AES-256-GCM krüpteerimist, Argon2id võtmetuletust ja HKDF domeenide eraldatust. Kliendipoolne rakendus on loodud React/TypeScript tehnoloogiatega, serveripoolne Node.js/Express raamistikuga. Testimine hõlmas 149 automatiseeritud testi, OWASP Top 10 vastavuse kontrolli, penetratsioonitestimist ja kasutatavuse testi viie osalejaga (SUS skoor 77,0). Kriitilisi turvanõrkusi ei tuvastatud.

Võtmesõnad: paroolihaldur, küberturve, krüpteerimine, veebirakendus, null-teadmise arhitektuur

CERCS: P170 Arvutiteadus, arvutusmeetodid, süsteemid, juhtimine (automaatjuhtimisteooria)

Development of a Secure Password Manager Web Application

Abstract:

This bachelor's thesis covers the design and development of a secure password manager web application. Passwords remain the most common authentication method, yet poor password management is one of the primary cybersecurity risks. The goal of this work is to create a zero-knowledge architecture-based password manager where all encryption and decryption occurs in the user's browser and the service provider never gains access to unencrypted data. The thesis analyzes existing password managers (Bitwarden, 1Password, LastPass, KeePass), describes the architecture and security model of the developed application, and presents the technical implementation. The application uses AES-256-GCM encryption, Argon2id key derivation, and HKDF for domain separation. The client-side application is built with React/TypeScript, the server-side with Node.js/Express. Testing comprised 149 automated tests, OWASP Top 10 compliance verification, penetration testing, and a usability test with five participants (SUS score 77.0). No critical security vulnerabilities were identified.

Keywords: password manager, cybersecurity, encryption, web application, zero-knowledge architecture, Argon2id, AES-256-GCM

CERCS: P170 Computer science, numerical analysis, systems, control

Sisukord

1. Sissejuhatus	7
2. Taustateadmised	9
2.1 Autentimise ja paroolide ajalugu	9
2.2 Paroolide turvalisuse probleemid	9
2.2.1 Paroolide korduvkasutamine	9
2.2.2 Nõrgad paroolid.....	10
2.2.3 Andmepüük ja sotsiaalse manipuleerimise rünnakud.....	10
2.2.4 Toore jõu ja sõnastikupõhised ründed	10
2.3 Krüptograafilised alused.....	11
2.3.1 Sümmeetriline ja asümmeetriline krüpteerimine.....	11
2.3.2 AES (Advanced Encryption Standard)	11
2.3.3 Toimingurežiimid: CBC vs GCM.....	11
2.3.4 Võtmetuletuse funktsioonid (KDF).....	12
2.3.5 Argon2 ja Argon2id	13
2.3.6 HKDF (HMAC-based Key Derivation Function)	13
2.3.7 Juhuslike arvude genereerimine	14
2.4 Null-teadmise arhitektuur	14
2.5 Veebiturvalisuse alused.....	15
2.5.1 Ristskriptimine (XSS).....	15
2.5.2 SQL-süst (SQL Injection).....	15
2.5.3 CSRF (Cross-Site Request Forgery)	15
2.5.4 Transpordi turvalisus	15
2.6 Olemasolevate lahenduste analüüs	15
2.6.1 Bitwarden.....	16
2.6.2 1Password	16
2.6.3 LastPass.....	16
2.6.4 KeePass	17
2.6.5 Lahenduste võrdlev kokkuvõte	17
2.7 Seotud tööd	17
3. Lahenduse kavandamine	19
3.1 Nõuded rakendusele	19
3.1.1 Funktsionaalsed nõuded	19

3.1.2	Mittefunktsionaalsed nõuded	20
3.2	Üldine arhitektuur	20
3.2.1	Kliendipoolne rakendus (frontend)	20
3.2.2	Serveripoolne rakendus (backend).....	21
3.2.3	Andmebaasiskeem	22
3.3	Turvamudel	23
3.3.1	Võtmehalduse skeem	23
3.3.2	Ohumudel ja vastumeetmed	25
3.3.3	Autentimisvoog	25
3.3.4	Sessiooniholdus ja automaatne lukustamine	26
3.4	API disain	27
3.5	Kasutajaliidese disain	27
4.	Realiseerimine.....	29
4.1	Arenduskeskkond ja tööriistad.....	29
4.2	Projekti struktuur	29
4.3	Kliendipoolne krüpteerimismoodul	30
4.3.1	Võtmetuletus Argon2id abil	30
4.3.2	Auth Key tuletus HKDF abil	31
4.3.3	AES-256-GCM krüpteerimine ja dekrüpteerimine	32
4.3.4	Encryption Key haldamine.....	33
4.3.5	Turvaline parooligenereerimine	34
4.4	Serveripoolne realiseerimine	35
4.4.1	Middleware kiht	35
4.4.2	Autentimise kontrollerr	35
4.4.3	Varahoidla kontrollerr.....	36
4.4.4	Andmebaasi migratsioonid	37
4.5	Kasutajaliidese realiseerimine	38
4.5.1	Olekuhaldus.....	39
4.5.2	Marsruutimine ja lehtede struktuur	39
4.5.3	Parooli tugevuse hindamine	39
4.5.4	Automaatne lukustamine	40
4.6	Arendusprotsess	40

5. Testimine ja tulemused	42
5.1 Funktsionaalne testimine	42
5.1.1 Krüpteerimismooduli ühiktestid	42
5.1.2 Serveripoolsed ühiktestid	44
5.1.3 Integratsioonitestid	44
5.2 Turvatestimine	46
5.2.1 Krüptograafiline analüüs	46
5.2.2 Veebiturvalisuse testimine	47
5.2.3 Penetratsioonitestimine	48
5.3 Jõudlustestimine	48
5.3.1 Kliendipoolne jõudlus	49
5.3.2 Serveripoolne jõudlus	49
5.4 Kasutatavuse testimine	49
5.4.1 Testi metoodika	49
5.4.2 Kasutatavuse testi tulemused	50
5.4.3 SUS tulemused	50
5.5 Testimise koondtulemused	51
6. Kokkuvõte	52
6.1 Peamised tulemused	52
6.2 Uurimisküsimustele vastamine	52
6.3 Piirangud	53
6.4 Edasiarendamise võimalused	54
Viited	55
Lisad	58
I. Lähtekoodi repositoorium	58
II. Paigaldusjuhend	58
III. Keskkonnamuutujad	59
IV. Testide käivitamine	59
V. Ekraanipildid	60
Litsents	61

1. Sissejuhatus

Tänapäeva digimaailmas on igapäevaelu lahutamatult seotud veebiteenustega. Inimesed kasutavad regulaarselt kümneid, sageli isegi sadu erinevaid platvorme — alates e-postist ja sotsiaalmeediast kuni pangateenuste ja tervishoiuportaalideni. Iga teenus nõuab eraldi kasutajakontot, mis tähendab, et keskmisel internetikasutajal on hallata märkimisväärne hulk parooli. Verizon'i 2023. aasta andmelekete raporti kohaselt on ligikaudu 80% andmeleketest seotud nõrkade, varastatud või korduvkasutatavate paroolidega [1]. See statistika rõhutab paroolihalduse kriitilist tähtsust kaasaegses küberturvalisuses.

Parooliturvalisuse probleemid ei ole uued, kuid nende ulatus kasvab pidevalt. Uuringud on näidanud, et keskmine kasutaja kasutab sama parooli keskmiselt viies erinevas teenuses [2]. Andmelekked paljastavad igal aastal miljardeid kasutajakontosid, mis tähendab, et ühe teenuse kompromiteerimine võib viia kõigi seotud kontode ülevõtmiseni [3]. Lisaks muutuvad paroolirünnakute meetodid üha keerukamaks: sõnastikupõhised ründed, *credential stuffing* (lekkinud paroolide massiline proovimine teistes teenustes) ja sotsiaalse manipuleerimise tehnikad on muutunud igapäevaseks.

Paroolihaldurid pakuvad sellele probleemile terviklikku lahendust, võimaldades kasutajatel genereerida ja salvestada unikaalseid, keerulisi parooli iga teenuse jaoks, meeles pidades vaid ühte peaparooli. Kuid paroolihalduri valik on ise oluline turvaotsus. Mitmete kaubanduslike lahenduste puhul on kasutaja sunnitud usaldama teenusepakkuja oma kõige tundlikumate andmetega. Viimaste aastate turvaprobleemid — näiteks LastPassi 2022. aasta andmeleke, kus ründajad said ligipääsu kasutajate krüpteeritud varahoidlatele [4] — on näidanud, et isegi tuntud teenusepakkujad ei pruugi olla piisavalt turvalised.

Null-teadmise arhitektuur (*zero-knowledge architecture*) on lähenemisviis, mis vähendab teenusepakkuja esitatavat usaldust miinimumini. Selle arhitektuuri puhul toimub kogu krüpteerimine ja dekrüpteerimine kasutaja seadmes ning teenusepakkuja ei saa kunagi ligipääsu kasutaja krüpteerimata andmetele. Isegi serveri täieliku kompromiteerimise korral jäävad kasutaja paroolid kaitstuks, kuna server salvestab ainult krüpteeritud andmeid ega oma võtit nende dekrüpteerimiseks.

Käesoleva bakalaureusetöö eesmärk on kavandada ja arendada turvaline paroolihalduri veebirakendus, mis põhineb null-teadmise arhitektuuril. Rakendus kasutab kaasaegseid krüptograafilisi algoritme — AES-256-GCM sümmeetrilist krüpteerimist ja Argon2id

võtmetulemust — ning on loodud avatud lähtekoodina, et tagada läbipaistvus ja auditeeritavus. Töö panustab valdkonda, pakkudes alternatiivset, Eesti konteksti arvestavat lahendust, mis on nii turvaline kui ka kasutatav.

Töö uurimisküsimused on järgmised:

1. Kuidas kavandada null-teadmise arhitektuuril põhinev paroolihaldur, mis tagab kasutajaandmete konfidentsiaalsuse isegi serveri kompromiteerimise korral?
2. Millised on kaasaegsed krüptograafilised parimad praktikad paroolihalduri arendamiseks?
3. Kuidas saavutada tasakaal turvalisuse ja kasutatavuse vahel paroolihalduri veebirakendusel?

Töö käigus analüüsitakse olemasolevaid paroolihaldureid ja nende turvaarhitektuure (peatükk 2). Peatükk 3 kirjeldatakse arendatava rakenduse arhitektuuri, turvamudelit ja kavandamisotsuseid. Peatükk 4 esitatakse tehniline realiseerimine koos koodinäidete ja implementatsiooniotsustega. Peatükk 5 hinnatakse rakenduse turvalisust, funktsionaalsust ja kasutatavust. Peatükk 6 tehakse kokkuvõtte tulemustest ning pakutakse välja edasiarendamise võimalused.

2. Taustateadmised

Käesolevas peatükis antakse ülevaade paroolihalduse valdkonnast, tutvustatakse olulisimaid krüptograafilisi mõisteid ja algoritme ning analüüsitakse olemasolevaid paroolihalduri lahendusi. Peatüki eesmärk on luua lugejale vajalik taustainfo, et mõista järgnevates peatükkides kirjeldatavaid arhitektuurilisi ja implementatsioonilisi otsuseid.

2.1 Autentimise ja paroolide ajalugu

Paroolipõhine autentimine on arvutisüsteemides kasutusel olnud juba alates 1960. aastatest, kui Fernando Corbató tutvustas paroolisüsteemi MIT Compatible Time-Sharing System (CTSS) jaoks [5]. Sellest ajast saadik on paroolid jäänud domineerivaks autentimismeetodiks, vaatamata mitmetele teadaolevatele nõrkustele.

Paroolide kasutamise algusaegadel olid süsteemid lihtsad: kasutajanimi ja parool salvestati lihttekstina. Morrise ja Thompsoni 1979. aasta töö [6] tutvustas paroolide räsimist ja soola kasutamist, mis kujunesid esimesteks olulisteks parooliturvalisuse meetmeteks. Need põhimõtted on olulised tänaseni, kuigi konkreetset algoritmid on oluliselt edasi arenenud.

Tänapäeval on paroolide kõrval levinud ka teised autentimismeetodid: biomeetria (sõrmejalg, näotuvastus), riistvaralised turvavõtmed (FIDO2/WebAuthn), ühekordse parooli generaatorid (TOTP) ja paroolita autentimine (*passwordless*). Sellegipoolest jääb paroolipõhine autentimine kõige levinumaks, kuna see ei nõua erivarustust ning on kasutajatele tuttav. FIDO Alliansi andmetel kasutab 2024. aastal endiselt üle 95% veebiteenustest paroolipõhist autentimist [7].

2.2 Paroolide turvalisuse probleemid

Paroolipõhise autentimise turvalisus sõltub fundamentaalselt kasutaja käitumisest, kuid inimeste kognitiivsed piirangud muudavad turvalise paroolikäitumise raskeks. Allpool käsitletakse peamisi probleemvaldkondi.

2.2.1 Paroolide korduvkasutamine

Florêncio ja Herley 2007. aasta ulatuslik uuring näitas, et keskmine kasutaja haldab 25 kontot, kuid kasutab vaid 6,5 unikaalset parooli [2]. See tähendab, et iga parooli kasutatakse keskmiselt neljas erinevas teenuses. Hilisemad uuringud on kinnitanud, et olukord ei ole oluliselt paranenud: Washingtoni Ülikooli 2019. aasta uuring leidis, et 52% kasutajatest kasutab sama parooli mitmes teenuses [8].

Korduvkasutamise oht ilmneb eriti selgelt *credential stuffing* rünnakute kontekstis. Kui üks teenus kogeb andmeleket, proovivad ründajad automaatselt lekkinud kasutajanime-parooli kombinatsioone teistes teenustes. Shape Security hinnangul moodustavad *credential stuffing* ründed kuni 80–90% veebipoodide sisselogimisliiklusest [9].

2.2.2 Nõrgad paroolid

Vaatamata teenusepakkujate kehtestatud paroolinõuetele (minimaalne pikkus, suurtähed, numbrid, erimärgid) valivad kasutajad sageli minimaalselt nõuetele vastavaid, kuid siiski ennustatavaid paroole. SplashData'i iga-aastased nimekirjad näitavad, et “123456”, “password” ja “qwerty” on püsinud enimlevinud paroolide hulgas üle kümne aasta [10].

Carnegei Melloni Ülikooli uurijad on näidanud, et traditsioonilistel koostisosanõuetel (nt “peab sisaldama suurtähte ja numbrit”) on väike mõju paroolide tegelikule entroopiale, kuna kasutajad kipuvad nõudeid täitma ennustatavatel viisidel — näiteks lisades suurtähe algusesse ja numbri lõppu [11]. Efektiivsemaks meetmeks on paroolide minimaalse pikkuse suurendamine ning lekkinud paroolide nimekirjadega võrdlemine.

2.2.3 Andmepüük ja sotsiaalse manipuleerimise rünnakud

Andmepüük (*phishing*) on rünnak, kus kasutajat meelitakse sisestama oma parooli võltsitud veebilehele. Google'i ja Californiya Ülikooli 2017. aasta uuring tuvastas ligikaudu 12,4 miljonit potentsiaalset andmepüügi ohvrit 12-kuulise perioodi jooksul [12]. Andmepüügi ründed muutuvad üha rafineeritumaks, kasutades tuntud brändide visuaalset identiteeti, domeenide sarnasust (*typosquatting*) ja isegi SSL-sertifikaate veenvuse lisamiseks.

Paroolihaldurid pakuvad andmepüügi vastu olulist kaitset, kuna automaatne paroolide täitmine põhineb URL-i täpsel vastavusel. Paroolihaldur ei täida parooli lehel, mille domeen erineb salvestatud kirjes olevast domeenist, mis aitab kasutajal märgata võltsitud veebilehti.

2.2.4 Toore jõu ja sõnastikupõhised ründed

Toore jõu ründe (*brute force attack*) korral proovib ründaja süstemaatiliselt kõiki võimalikke paroole. Sõnastikupõhine rünne kasutab sõnaloendeid ja levinud paroole, et vähendada otsinguruumi. Kaasaegsete GPU-de ja spetsialiseeritud riistvara (nt ASIC) abil on võimalik proovida miljardeid parooli sekundis [13].

Kaitse toore jõu rünnakute vastu põhineb kahel strateegial. Esiteks, paroolide entroopia suurendamine (pikemad ja juhuslikumad paroolid). Teiseks, võtmetuletuse funktsioonide (KDF) kasutamine, mis muudavad iga üksiku parooli kontrollimise arvutuslikult kulukaks. Viimane

strateegia on eriti oluline paroolihaldurite kontekstis, kus peaparooli ründamine on peamine ohustenaarium.

2.3 Krüptograafilised alused

Turvalise paroolihalduri ehitamiseks on vaja mõista mitut krüptograafilist primitiivi. Käesolevas alampeatükis tutvustatakse paroolihalduri jaoks olulisi krüptograafilisi kontseptsioone ja algoritme.

2.3.1 Sümmeetriline ja asümmeetriline krüpteerimine

Krüpteerimine jaguneb kaheks põhitüübiks. Sümmeetrilise krüpteerimise puhul kasutatakse sama võtit nii krüpteerimiseks kui dekrüpteerimiseks. Asümmeetrilise krüpteerimise puhul kasutatakse võtmepaari: avalikku võtit krüpteerimiseks ja privaatvõtit dekrüpteerimiseks [14].

Paroolihaldurid kasutavad peamiselt sümmeetrilist krüpteerimist, kuna see on asümmeetrilisest märkimisväärselt kiirem ja sobib hästi suurte andmehulkade krüpteerimiseks. Asümmeetrilist krüpteerimist kasutatakse paroolihaldurites pigem spetsiifilistes stsenaariumites, näiteks turvalise paroolide jagamise funktsionaalsuse realiseerimiseks.

2.3.2 AES (Advanced Encryption Standard)

AES on 2001. aastal NIST-i poolt standardiseeritud sümmeetrilise krüpteerimise algoritm [15]. AES põhineb Rijndaeli plokkšifril ning toetab kolme võtmepikkust: 128, 192 ja 256 bitti. AES-256 kasutab 14 raundi transformatsioone (SubBytes, ShiftRows, MixColumns, AddRoundKey) ning pakub 256-bitist turvataset.

AES on kõige laialdasemalt kasutatav sümmeetrilise krüpteerimise algoritm maailmas. Seda kasutavad nii valitsusasutused (NSA on tunnistanud AES-256 sobivaks *Top Secret* tasemel info kaitsmiseks) kui ka enamik kaubanduslikke turvarakendusi. AES-i turvalisus on ulatuslikult analüüsitud ning tänaseni ei ole leitud praktilist rünnakut, mis oleks efektiivsem kui toore jõu ründe.

2.3.3 Toimingurežiimid: CBC vs GCM

Plokkšifrid nagu AES krüpteerivad andmeid fikseeritud suurusega plokkidena (AES puhul 128 bitti). Pikemate andmete krüpteerimiseks kasutatakse toimingurežiime (*modes of operation*), mis määravad, kuidas plokkide krüpteerimine omavahel seotakse.

CBC (Cipher Block Chaining) on traditsiooniline toimingurežiim, kus iga plokk krüpteeritakse eelmise krüpteeritud ploki XOR-iga. CBC pakub konfidentsiaalsust, kuid mitte andmete

tervkluse kontrolli. Bitwarden kasutab AES-256-CBC koos eraldi HMAC-SHA256 autentimisega [16].

GCM (*Galois/Counter Mode*) on autenditud krüpteerimise režiim, mis ühendab Counter Mode (CTR) krüpteerimise Galois räsimisega. GCM pakub üheaegselt nii konfidentsiaalsust kui ka andmete tervikluse kontrolli (autentsust), mis tähendab, et andmete muutmine on tuvastatav ilma eraldi HMAC-i arvutamiset [17]. See teeb GCM-i CBC-st eelistatumaks, kuna vähendab implementatsiooni keerukust ja potentsiaalsete vigade arvu.

Käesolev töö kasutab AES-256-GCM režiimi, kuna see pakub nii krüpteerimist kui autentimist ühe operatsiooniga, on hästi toetatud Web Crypto API poolt ning on laialdaselt tunnustatud parimaks praktikaks.

2.3.4 Võtmetuletuse funktsioonid (KDF)

Võtmetuletuse funktsioon (*Key Derivation Function*, KDF) on krüptograafiline primitiiv, mis tuletab ühest sisendvõtmest (tavaliselt paroolist) ühe või mitu krüptograafiliselt turvalist võtit. Paroolihaldurite kontekstis on KDF-i ülesanne muuta kasutaja peaparool (mis on tavaliselt madala entroopiaga) kõrge entroopiaga krüpteerimisvõtmeks [18].

Hea paroolipõhine KDF peab olema tahtlikult aeglane, et muuta toore jõu ründed arvutuslikult kulukaks. Ajalooliselt on selleks kasutatud järgmisi funktsioone:

PBKDF2 (*Password-Based Key Derivation Function 2*) on 2000. aastal standardiseeritud KDF [19], mis kasutab HMAC funktsiooni korduvat rakendamist. PBKDF2 on laialdaselt toetatud ja standardiseeritud, kuid selle peamine nõrkus on paralleliseeritavus: GPU-d ja ASIC-d saavad PBKDF2 arvutust väga efektiivselt paralleliseerida, mis vähendab selle vastupidavust toore jõu rünnete.

bcrypt on 1999. aastal loodud parooliräsamise funktsioon [20], mis põhineb Blowfishi šifril. Bcrypt on mälumahukam kui PBKDF2, kuid selle mälunõue on fikseeritud 4 KB juures, mis ei ole kaasaegsete rünnakute kontekstis piisav.

scrypt on 2009. aastal loodud KDF [21], mis tutvustas seadistava mälumahukuse kontseptsiooni. Scrypt nõuab suure hulga mälu kasutamist, mis muudab rünnakud spetsialiseeritud riistvaraga (GPU, ASIC) märkimisväärselt kulukamaks. Scrypti puuduseks on aga raskesti valideeritav turvalisus ja mõnede implementatsioonide probleemsus.

2.3.5 Argon2 ja Argon2id

Argon2 on 2015. aasta Password Hashing Competition (PHC) võitja, mille lõi Alex Biryukov, Daniel Dinu ja Dmitry Khovratovich Luksemburgi Ülikoolist [22]. Argon2 eesmärk oli lahendada varasemate KDF-ide puudused ja pakkuda optimaalset kaitset mitmesuguste rünnakuvektorite vastu.

Argon2-l on kolm varianti:

- **Argon2d** — andmest sõltuv mälupeerdumismuster. Pakub tugevamat kaitset GPU rünnakute vastu, kuid on haavatav ajapõhiste kõrvalkanalite (*timing side-channel*) rünnakute suhtes.
- **Argon2i** — andmetest sõltumatu mälupeerdumismuster. Vastupidav ajapõhiste kõrvalkanalite rünnakutele, kuid nõrgem GPU rünnakute vastu.
- **Argon2id** — hübriidvariant, mis ühendab mõlema eelised. Esimene pool iteratsioonist kasutab Argon2i mustrit (kõrvalkanalite kaitse) ja teine pool Argon2d mustrit (GPU kaitse).

Argon2id on OWASP-i soovitatud vaikevalik paroolide räsimiseks [23]. Funktsioon võtab sisendiks parooli, soola ning kolm parameetrit:

- **Mälu** (m) — kasutatava mälu hulk kilobaitides. Suurem mälu kasutus muudab ründe kulukamaks.
- **Iteratsioonid** (t) — ajalise kulu parameeter. Suurem väärtus suurendab arvutusaega.
- **Paralleelsus** (p) — kasutatavate lõimede arv.

OWASP soovib minimaalseteks parameetriteks: mälu 47 MiB ($m = 47104$), iteratsioonid 1 ($t = 1$), paralleelsus 1 ($p = 1$), või alternatiivina mälu 19 MiB ($m = 19456$), iteratsioonid 2 ($t = 2$), paralleelsus 1 ($p = 1$) [23]. Käesolevas töös kasutatakse veidi tugevamaid parameetreid, mida kirjeldatakse peatükis 3.

2.3.6 HKDF (HMAC-based Key Derivation Function)

HKDF on RFC 5869-ga standardiseeritud võtmetuletuse funktsioon [24], mis tuletab ühest võtmest mitu tuletatud võtit. HKDF koosneb kahest sammust: *extract* (entroopia kontsentreerimine) ja *expand* (võtmematerjali laiendamine).

Paroolihalduri kontekstis kasutatakse HKDF-i peamiselt selleks, et tuletada peaparoolist (*Master Key*) eraldi võtmed erinevate eesmärkide jaoks: üks võti andmete krüpteerimiseks ja teine autentimiseks. See tagab, et autentimiseks serverile saadetakse võti ei paljasta krüpteerimisvõtit ja vastupidi.

2.3.7 Juhuslike arvude genereerimine

Krüptograafiliselt turvalised juhuslikud arvud on iga turvasüsteemi alustala. Paroolihaldur vajab juhuslikke arve initialiseerimise vektorite (IV), soolade ja genereeritud paroolide jaoks. Ebaturvalise juhuslike arvude generaatori kasutamine võib nullida kogu ülejäänud krüptograafia turbe [14].

Veebibrauserites pakub turvalisi juhuslikke arve `crypto.getRandomValues()` meetod (Web Crypto API osa), mis kasutab operatsioonisüsteemi krüptograafiliselt turvalist juhuslike arvude generaatorit (CSPRNG). Node.js serveris on analoogne funktsioon `crypto.randomBytes()`.

2.4 Null-teadmise arhitektuur

Null-teadmise arhitektuur (*zero-knowledge architecture*) on turvaarhitektuuriline põhimõte, kus teenusepakkuja ei saa kunagi ligipääsu kasutaja krüpteerimata andmetele. Termin “null-teadmine” on paroolihaldurite kontekstis laenatud null-teadmise tõestuste (*zero-knowledge proofs*) mõistest krüptograafias, kuigi tehniline implementatsioon on erinev [25].

Null-teadmise arhitektuuri peamised põhimõtted on:

1. **Kliendipoolne krüpteerimine.** Kogu andmete krüpteerimine ja dekrüpteerimine toimub kasutaja seadmes. Server ei teosta krüpteerimisoperatsioone kasutaja andmetega.
2. **Võtmete lokaalsus.** Krüpteerimisvõtmed ei lahku kunagi kasutaja seadmest. Serverile saadetakse ainult autentimiseks vajalik info, mis on krüpteerimisvõtmest matemaatiliselt tuletamatu.
3. **Minimaalne usaldus.** Kasutaja ei pea usaldama teenusepakkujat. Isegi pahatahtlik server ei saa ligipääsu kasutaja andmetele.
4. **End-to-end krüpteerimine.** Andmed on krüpteeritud kogu nende elutsükli jooksul, välja arvatud ajal, mil kasutaja neid aktiivselt vaatab.

Null-teadmise arhitektuuril on ka piiranguid. Kuna server ei saa andmeid dekrüpteerida, ei saa ta pakkuda serveripoolset otsingut, andmete indekseerimist ega taastamist peaparooli kaotamise

korral. Need piirangud mõjutavad otseselt rakenduse funktsionaalsust ja kasutajakogemust ning neid tuleb kavandamisel arvestada.

2.5 Veebiturvalisuse alused

Paroolihalduri veebirakenduse arendamisel tuleb arvestada lisaks krüptograafiale ka üldiste veebiturvalisuse aspektidega. OWASP (*Open Web Application Security Project*) Top 10 nimekiri defineerib kõige kriitilisemad veebiturvalisuse riskid [26].

2.5.1 Ristskriptimine (XSS)

Ristskriptimine (*Cross-Site Scripting*, XSS) on rünnak, kus pahatahtlik JavaScript-kood süstitakse veebilehele ja käivitatakse kasutaja brauseris [26]. Paroolihalduri kontekstis on XSS eriti ohtlik, kuna see võib anda ründajale ligipääsu dekrüpteeritud andmetele brauseri mälus.

Kaitse XSS vastu hõlmab sisendi valideerimist ja puhastamist, väljundi kodeerimist, Content Security Policy (CSP) päiste kasutamist ning HttpOnly küpsiste rakendamist. React raamistik pakub vaikinisi XSS kaitset, kuna JSX väljund on automaatselt HTML-kodeeritud.

2.5.2 SQL-süst (SQL Injection)

SQL-süst on rünnak, kus pahatahtlik SQL-kood sisestatakse rakenduse päringusse, võimaldades andmebaasi volitamata lugemist, muutmist või kustutamist. Kaitse hõlmab parameetritega päringute kasutamist (ORM-i kaudu) ning sisendi valideerimist.

2.5.3 CSRF (Cross-Site Request Forgery)

CSRF rünnak sunnib kasutaja brauserit tegema soovimatuid päringuid teenusele, kuhu kasutaja on sisse logitud. Kaitse hõlmab CSRF-tokenite, SameSite küpsise atribuudi ja päritolupäise kontrolli kasutamist.

2.5.4 Transpordi turvalisus

HTTPS (HTTP over TLS) tagab andmete konfidentsiaalsuse ja tervikluse transpordikihis. Paroolihalduri puhul on HTTPS hädavajalik, kuna ilma selleta saaks vahemehe rünnakuga pealt kuulata kogu suhtlust kliendi ja serveri vahel, sealhulgas autentimistokeneid.

2.6 Olemasolevate lahenduste analüüs

Turul on mitmeid paroolihaldureid, millest igaühel on oma eelised ja puudused. Käesolevas alampeatükis analüüsitakse nelja levinumat lahendust nende turvaarhitektuuri ja funktsionaalsuse seisukohalt.

2.6.1 Bitwarden

Bitwarden on 2016. aastal loodud avatud lähtekoodiga paroolihaldur, mis on saanud kiiresti populaarseks eriti tehnikateadlike kasutajate seas. Bitwardeni turvaarhitektuur põhineb null-teadmise mudelil [16].

Bitwardeni krüptograafiline skeem toimib järgmiselt. Kasutaja peaparoolist tuletatakse PBKDF2-SHA256 abil (vaikimisi 600 000 iteratsiooni) *Master Key*. Sellest tuletatakse omakorda *Master Password Hash* autentimiseks ning *Stretched Master Key* (HKDF-SHA256 abil) krüpteerimiseks. Igal kasutajal on *Symmetric Key*, mis on krüpteeritud *Stretched Master Key* abil ja salvestatud serveris. Kasutaja andmed krüpteeritakse AES-256-CBC + HMAC-SHA256 abil.

Bitwardeni peamine tugevus on avatud lähtekood, mis võimaldab sõltumatut turvaauditit. 2023. aastal teostas Cure53 Bitwardeni turvaauditi, mille tulemused on avalikult kättesaadavad [27]. Peamine nõrkus on PBKDF2 kasutamine, mis on mälupõhiste rünnakute vastu nõrgem kui Argon2 (kuigi Bitwarden on hiljem lisanud Argon2id toe valikulise seadistusena).

2.6.2 1Password

1Password on 2006. aastal loodud kommertsiaalne paroolihaldur, mida arendab kanada ettevõtte AgileBits [28]. 1Password eristub teistest unikaalse *Secret Key* süsteemiga.

1Passwordi turvamudel kasutab kahte sõltumatut salajast väärtust: peaparooli ja *Secret Key* (128-bitine juhuslik väärtus, mis genereeritakse konto loomisel). Krüpteerimisvõti tuletatakse mõlema väärtuse kombinatsioonist, mis tähendab, et isegi peaparooli teadmine ei ole piisav andmete dekrüpteerimiseks. See pakub tugevat kaitset serveripoolse andmelekkete vastu, kuna *Secret Key* salvestatakse ainult kasutaja seadmes, mitte serveris.

1Passwordi nõrkuseks on kinnine lähtekood, mis muudab sõltumatu turvaauditi raskeks. Lisaks on *Secret Key* süsteem kasutajale keerulisem: võtit tuleb kindlasti varundada, kuna selle kaotamisel on andmete taastamine võimatu.

2.6.3 LastPass

LastPass on üks vanimaid ja varem populaarsemaid pilvepõhiseid paroolihaldureid. LastPassi turvaarhitektuur kasutab samuti null-teadmise mudelit, kus peaparoolist tuletatakse PBKDF2-SHA256 abil krüpteerimisvõti [29].

LastPassi juhtum on käesoleva töö kontekstis eriti õpetlik. 2022. aasta augustis ja novembris kogeti kahte suurt turvaintsidenti, kus ründajad said ligipääsu arenduskeskkonnale ja seejärel

pilvevarundustele, mis sisaldasid kasutajate krüpteeritud varahoidlaid [4]. Kuigi varahoidlad olid krüpteeritud, muutis PBKDF2 suhteliselt madal iteratsioonide arv (paljudel vanematel kontodel vaid 5 000 iteratsiooni) nõrkade peaparoolide murdmise teoreetiliselt teostatavaks.

See intsident rõhutab mitut olulist aspekti: tugevama KDF-i (nagu Argon2id) tähtsust, serveri turvalisuse kriitilisust (isegi null-teadmise arhitektuuriga) ning vanade kontode parameetrite uuendamise vajadust.

2.6.4 KeePass

KeePass on 2003. aastal loodud lokaalne, avatud lähtekoodiga paroolihaldur, mis salvestab kõik andmed ühes krüpteeritud andmebaasifailis (.kdbx) [30]. Erinevalt eelmistest lahendustest ei kasuta KeePass pilveserverit — andmed salvestatakse ainult kasutaja seadmes.

KeePassi krüptograafiline skeem kasutab AES-256 või ChaCha20 krüpteerimist. Võtmetuletus kasutab kas Argon2d (alates versioonist 2.x) või AES-KDF funktsiooni. KeePass toetab ka võtmefaile (*key files*) ja Windowsi kasutajakonto seostamist (*Windows User Account*), mis pakuvad täiendavat turvakihti.

KeePassi peamine eelis on täielik kontroll andmete üle — andmed ei lahku kunagi kasutaja seadmest (välja arvatud kui kasutaja ise sünkroniseerib faili pilveteenuse kaudu). Peamine puudus on sünkroniseerimise keerukus: mitmest seadmest kasutamine nõuab kolmanda osapoole pilveteenuse (nt Dropbox, Google Drive) seadistamist ja konfliktide käsitsi lahendamist.

2.6.5 Lahenduste võrdlev kokkuvõte

Tabel 1 esitab olemasolevate lahenduste peamised omadused ja võrdleb neid käesolevas töös arendatava rakendusega.

Tabelist 1 on näha, et käesoleva töö rakendus eristub Argon2id vaikimisi kasutamise, AES-256-GCM autenditud krüpteerimise ja täieliku tasuta avatud lähtekoodi poolest. Kuigi olemasolevad lahendused on küpsemad ja funktsionaalselt rikkamad, pakub käesolev töö kaasaegsemat krüptograafilist alust.

2.7 Seotud tööd

Paroolihalduse ja parooliturvalisuse teemadel on avaldatud mitmeid akadeemilisi töid. Li et al. [31] analüüsisid paroolihaldurite kasutatavust ja leidsid, et isegi tehnikateadlikud kasutajad teevad turvakriitilisi vigu paroolihaldurite kasutamisel. Oesch ja Ruoti [32] uurisid

Tabel 1. Olemasolevate paroolihaldurite võrdlus.

Omadus	Bitwarden	1Password	KeePass	Käesolev töö
Avatud lähtekood	Jah	Ei	Jah	Jah
Null-teadmine	Jah	Jah	Jah*	Jah
Veebiliides	Jah	Jah	Ei	Jah
Tasuta	Osaliselt	Ei	Jah	Jah
Vaikimisi KDF	PBKDF2	PBKDF2	Argon2d	Argon2id
Krüpteerimine	AES-256-CBC	AES-256-GCM	AES-256/ChaCha20	AES-256-GCM
Pilvepõhine	Jah	Jah	Ei	Jah

*KeePass on lokaalne, seega null-teadmine on sisuliselt tagatud arhitektuuri kaudu.

brauseripõhiste paroolihaldurite turvamudeleid ja tuvastasid mitmeid potentsiaalseid haavatavusi, eriti seoses automaatse täitmise funktsiooniga.

Stock ja Johns [33] uurisid paroolihaldurite vastupidavust andmepüügi rünnakutele ning leidsid, et brauseripõhised lahendused pakuvad paremat kaitset kui manuaalne paroolihaldus, kuid ei ole immuunsed keerulisemate rünnakute vastu.

Käesolev töö erineb nimetatud uuringutest, keskendudes konkreetse null-teadmise arhitektuuril põhineva paroolihalduri kavandamisele ja arendamisele, kasutades kaasaegsemaid krüptograafilisi algoritme (Argon2id, AES-256-GCM) kui enamik olemasolevaid lahendusi.

3. Lahenduse kavandamine

Käesolevas peatükis kirjeldatakse arendatava paroolihalduri veebirakenduse nõudeid, arhitektuuri, turvamudelit ja kasutatavaid tehnoloogiaid. Kavandamisotsused põhinevad peatükis 2 esitatud analüüsil olemasolevate lahenduste tugevustest ja nõrkustest.

3.1 Nõuded rakendusele

Nõuete defineerimine on tarkvaraarenduse esimene ja kriitilise tähtsusega etapp. Nõuded jagunevad funktsionaalseteks (mida rakendus peab tegema) ja mittefunktsionaalseteks (kuidas rakendus peab töötama).

3.1.1 Funktsionaalsed nõuded

Funktsionaalsed nõuded on tuletatud olemasolevate paroolihaldurite analüüsist ja parimate praktikate ülevaatest:

1. **FN1: Kasutaja registreerimine.** Kasutaja saab luua konto e-posti aadressi ja peaparooliga. Registreerimisel kuvatakse peaparooli tugevuse hinnang.
2. **FN2: Sisselogimine.** Kasutaja saab sisse logida peaparooli abil. Rakendus toetab sisselogimiskatsete piirangut.
3. **FN3: Paroolide haldamine.** Kasutaja saab luua, vaadata, muuta ja kustutada paroolide kirjeid. Iga kirje sisaldab vähemalt: pealkirja, kasutajanime, parooli ja URL-i.
4. **FN4: Turvaline parooligenereerimine.** Kasutaja saab genereerida krüptograafiliselt turvalisi juhuslikke paroole seadistatavate parameetritega (pikkus, märgitüübid).
5. **FN5: Otsing ja filtreerimine.** Kasutaja saab otsida salvestatud kirjeid pealkirja, kasutajanime ja URL-i järgi.
6. **FN6: Kaustade tugi.** Kasutaja saab organiseerida kirjeid kaustadesse hierarhilise struktuuri abil.
7. **FN7: Märkmete tugi.** Kasutaja saab lisada kirjetele krüpteeritud märkmeid vabatekstina.
8. **FN8: Paroolide eksport ja import.** Kasutaja saab eksportida oma andmed krüpteeritud ja krüpteerimata (CSV) kujul ning importida andmeid levinud paroolihaldurite formaadis.
9. **FN9: Peaparooli vahetamine.** Kasutaja saab vahetada peaparooli ilma andmeid kaotamata (ümberkrüpteerimine).

10. **FN10: Turvaline lõpetamine.** Rakendus lukustatakse automaatselt pärast seadistatavat tegevusetuse perioodi.

3.1.2 Mittefunktsionaalsed nõuded

1. **MFN1: Null-teadmise arhitektuur.** Kogu kasutajaandmete krüpteerimine ja dekrüpteerimine toimub kliendi poolel. Server ei saa kunagi ligipääsu krüpteerimata andmetele ega krüpteerimisvõtmetele.
2. **MFN2: Kaasaegne krüptograafia.** Rakendus kasutab AES-256-GCM krüpteerimist ja Argon2id võtmetuletust vastavalt OWASP-i soovitudele.
3. **MFN3: Brauserite ühilduvus.** Rakendus peab töötama kõigis kaasaegsetes brauserites (Chrome, Firefox, Safari, Edge) viimase kahe versiooni ulatuses.
4. **MFN4: Jõudlus.** Võtmetuletus peab olema teostav alla 3 sekundi keskmise jõudlusega seadmel. Varahoidla dekrüpteerimine kuni 1000 kirjega peab toimuma alla 2 sekundi.
5. **MFN5: Skaleeritavus.** Server peab taluma vähemalt 100 samaaegset kasutajat ilma märgatava jõudluse languseta.
6. **MFN6: Kättesaadavus.** Rakendus peab olema mobiilsõbralik (*responsive design*).
7. **MFN7: Avatud lähtekood.** Kogu rakenduse lähtekood on avalikult kättesaadav auditeerimiseks.

3.2 Üldine arhitektuur

Rakendus järgib klassikalist kolmekihilist klient-server arhitektuuri, mis koosneb kliendipoolsest rakendusest (*frontend*), serveripoolsest rakendusest (*backend*) ja andmebaasist. Joonis 1 annab ülevaate süsteemi komponentidest ja nende omavahelistest seostest.

Oluline on mõista, et krüpteerimise piir (*trust boundary*) asub kliendi ja serveri vahel. Kõik andmed, mis ületavad selle piiri kliendilt serverile, on juba krüpteeritud. Server ja andmebaas käitlevad ainult krüpteeritud andmeid ning neil puudub võimekus andmeid dekrüpteerida.

3.2.1 Kliendipoolne rakendus (frontend)

Kliendipoolne rakendus on üheleheline veebirakendus (*Single Page Application*, SPA), mis vastutab kasutajaliidese kuvamise ning kogu krüpteerimis- ja dekrüpteerimisloogika eest. Tehnoloogiavirna valik on tehtud järgmistel kaalutlustel:

Siia tuleb arhitektuuridiagramm, mis näitab kliendi, serveri ja andmebaasi suhtlust ning krüpteerimise piiri.

Joonis 1. Süsteemi üldine arhitektuur.

React on valitud kasutajaliidese raamistikuks selle laialdase kogukonna toe, küpse ökosüsteemi ja komponentpõhise arhitektuuri tõttu. React võimaldab kasutajaliidest jagada taaskasutatavarteks komponentideks, mis hõlbustab koodi hooldatavust ja testimist.

TypeScript on valitud JavaScripti asemel, kuna staatiline tüübikontroll vähendab arendusvigade riski ja parandab koodi dokumenteeritust. Turvakriitilises rakenduses on tüübikindlus eriti oluline, kuna see aitab vältida andmetüüpidega seotud haavatavusi.

Web Crypto API on brauseri natiivne krüptograafiateek, mis pakub standardiseeritud ja riistvara kiirendusega krüptograafilisi operatsioone. Web Crypto API eelis kolmandate osapoolte teekide ees on see, et operatsioonid teostatakse brauseri turvalises keskkonnas (*secure context*) ning võtmematerjalile ei ole JavaScriptist otsest ligipääsu (`CryptoKey` objektid on mitteekstraheeritavad).

Tailwind CSS on valitud stiiliraamistikuks, kuna utiliidipõhine lähenemine võimaldab kiiret arendust ja ühtlast visuaalset keelt ilma eraldi CSS-failideta.

3.2.2 Serveripoolne rakendus (backend)

Serveripoolne rakendus haldab kasutajate autentimist, krüpteeritud andmete salvestamist ja API päringute töötlemist. Oluline on rõhutada, et server **ei teosta** krüpteerimist ega dekrüpteerimist kasutaja andmetega.

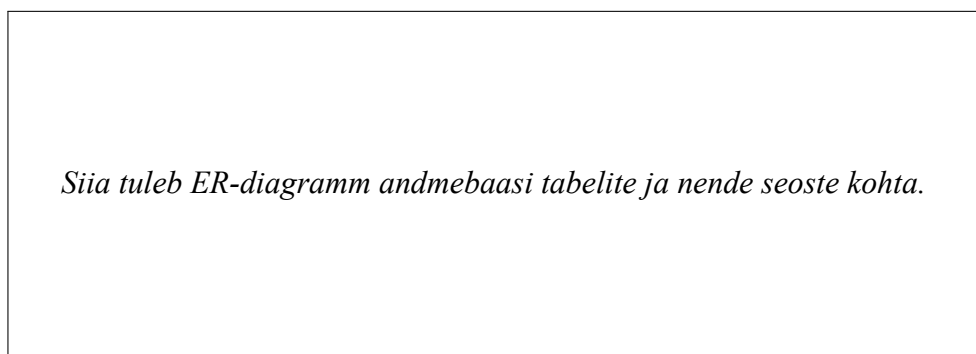
Node.js koos **Express** raamistikuga on valitud serveripoolseks platvormiks. Node.js pakub kiiret asünkroonset I/O mudelit, mis sobib hästi API päringute töötlemiseks. Express on minimalistlik ja paindlik raamistik, mis ei kehtesta liigset struktuuri. Ühtne JavaScripti/TypeScripti kasutamine nii kliendis kui serveris lihtsustab arendust ja vähendab kontekstivahetuse kulu.

PostgreSQL on valitud andmebaasiks selle töökindluse, ACID vastavuse ja laialdaste turvaomaduste tõttu. PostgreSQL toetab ka JSON-andmetüüpi, mis on kasulik krüpteeritud andmete metainfo salvestamiseks.

Prisma ORM on valitud andmebaasikihiks, kuna see pakub tüübikindlat andmebaasiliidest TypeScripti jaoks, automaatset skeemimigratsioonide haldamist ja kaitset SQL-süstide vastu.

3.2.3 Andmebaasiskeem

Andmebaasiskeem on kavandatud minimalistlikult, salvestades ainult hädavajaliku info. Joonis 2 esitab andmebaasi olemi-suhte diagrammi.



Joonis 2. Andmebaasi olemi-suhte diagramm.

Andmebaas sisaldab järgmisi põhitabeleid:

users tabel salvestab kasutajakonto andmed:

- `id` — unikaalne identifikaator (UUID).
- `email` — kasutaja e-posti aadress (unikaalne, indekseeritud).
- `auth_hash` — autentimISRäsi (Argon2id räsi serveripoolse soolaga).
- `kdf_salt` — soola väärtus kliendipoolse võtmetuletuse jaoks.
- `kdf_iterations` — Argon2id iteratsioonide arv.
- `kdf_memory` — Argon2id mälu kasutus.
- `encrypted_key` — krüpteeritud sümmeetriline võti (krüpteeritud *Master Key*-ga).
- `created_at`, `updated_at` — ajatemplid.

vault_items tabel salvestab krüpteeritud kirjed:

- `id` — unikaalne identifikaator.

- `user_id` — viide kasutajale (võõrvõti).
- `folder_id` — viide kaustale (valikuline).
- `type` — kirje tüüp (parool, märkus, kaart).
- `encrypted_data` — krüpteeritud andmed (JSON, AES-256-GCM).
- `iv` — initialiseerimise vektor.
- `created_at`, `updated_at` — ajatemplid.

folders tabel salvestab kaustade struktuuri:

- `id` — unikaalne identifikaator.
- `user_id` — viide kasutajale.
- `encrypted_name` — krüpteeritud kausta nimi.
- `iv` — initialiseerimise vektor.
- `parent_id` — viide vanemkaustale (hierarhia jaoks, valikuline).

Oluline on märkida, et andmebaasis ei salvestata ühtegi lihttekstina kasutajaga seotud andmestikku peale e-posti aadressi (mis on vajalik sisselogimiseks) ja tehniliste metaandmete (ajatemplid, ID-d).

3.3 Turvamudel

Turvamudel defineerib, kuidas rakendus kaitseb kasutaja andmeid erinevate ohtude vastu. Mudel põhineb null-teadmise arhitektuuril ja mitmekihilisel kaitsel (*defense in depth*).

3.3.1 Võtmehalduse skeem

Rakenduse võtmehaldus on kavandatud nii, et server ei saa kunagi ligipääsu kasutaja krüpteerimisvõtmetele. Joonis 3 illustreerib võtmetuletuse ja krüpteerimise protsessi.

Võtmehalduse skeem koosneb järgmistest sammudest:

Samm 1: Master Key tuletus. Kasutaja peaparoolist ja kasutaja e-posti aadressist (soolana) tuletatakse Argon2id abil 256-bitine *Master Key*. E-posti kasutamine soolana tagab, et sama parooliga eri kasutajatel on erinevad *Master Key* väärtused. Argon2id parameetrid on:

- Mälu: 64 MiB ($m = 65536$)

Siia tuleb diagramm, mis näitab võtmetuletuse ahelat: peaparool $\rightarrow$ Master Key $\rightarrow$ Auth Key / Encryption Key.

Joonis 3. Võtmetuletuse ja krüpteerimise skeem.

- Iteratsioonid: 3 ($t = 3$)
- Paralleelsus: 4 ($p = 4$)
- Väljundi pikkus: 256 bitti

Need parameetrid ületavad OWASP-i soovitatud miinimumtasemed [23] ja tagavad, et ühe parooli katsetamine võtab rünnaja seadmel vähemalt mõnisada millisekundit.

Samm 2: Auth Key tuletus. *Master Key*-st tuletatakse HKDF-SHA256 abil *Auth Key*, mis on mõeldud ainult autentimiseks. HKDF kontekstiinfo (`info` parameeter) on seatud väärtusele `auth-key`, mis tagab domeenide eraldatuse.

Samm 3: Serveripoolne autentimisräsi. Server arvutab saadud *Auth Key*-st täiendava Argon2id räsi serveripoolse juhusiku soolaga. See tagab, et isegi andmebaasi lekke korral ei saa ründaja otse kasutada *Auth Key* väärtust sisselogimiseks.

Samm 4: Encryption Key. Registreerimisel genereeritakse kliendipoolselt 256-bitine juhuslik *Encryption Key*. See võti krüpteeritakse *Master Key* abil (AES-256-GCM) ja saadetakse serverile salvestamiseks. Sisselogimisel laetakse krüpteeritud *Encryption Key* serverist ja dekrüpteeritakse kliendipoolselt *Master Key* abil.

Samm 5: Andmete krüpteerimine. Kõik kasutaja andmed (paroolid, märkmed, kaustanimed) krüpteeritakse *Encryption Key* abil AES-256-GCM režiimis. Iga krüpteerimisoperatsioon kasutab unikaalset 96-bitist juhuslikku IV-d.

Kahetasemeline võtmeskeem (*Master Key* krüpteerib *Encryption Key*, mis omakorda krüpteerib andmeid) on oluline, kuna see võimaldab peaparooli vahetamist ilma kõigi andmete ümberkrüpteerimiseta — vahetada tuleb ainult *Encryption Key* krüpteering uue *Master Key*-ga.

3.3.2 Ohumudel ja vastumeetmed

Ohumudel kirjeldab potentsiaalseid ründeid ja nende vastu rakendatavaid vastumeetmeid. Tabel 2 esitab peamised ohud ja vastumeetmed.

Tabel 2. Ohumudel ja vastumeetmed.

Oht	Tõsidus	Vastumeetmed
Serveri andmebaasi leke	Kõrge	Null-teadmise arhitektuur; Argon2id autentimisräsi; andmed krüpteeritud AES-256-GCM
Vahemehe rünnak (MITM)	Kõrge	HTTPS/TLS; HSTS päis; sertifikaatide kinnistamine
Ristskriptimine (XSS)	Kriitiline	React automaatne kodeerimine; CSP päised; sisendi sanitiseerimine
Toore jõu rünnak peaparoolile	Kõrge	Argon2id (aeglane KDF); sisselogimispiirang; konto lukustamine
CSRF	Keskmine	SameSite küpsis; CSRF-tokenid; päritolupäise kontroll
SQL-süst	Kõrge	Prisma ORM (parameetritega päringud); sisendi valideerimine
Kliendipoolne mälurünnak	Keskmine	Võtmete puhastamine pärast kasutamist; automaatne lukustamine
Seansi kaaperdamine	Kõrge	JWT lühike kehtivusaeg; HttpOnly/Secure küpsised

3.3.3 Autentimisvoog

Autentimisvoog on kavandatud nii, et peaparool ega *Master Key* ei saadeta kunagi serverile. Allpool kirjeldatakse registreerimise ja sisselogimise protsessi.

Registreerimise voog:

1. Kasutaja sisestab e-posti ja peaparooli.
2. Klient kontrollib peaparooli tugevust (minimaalselt 10 märki; zxcvbn teegi abil hinnatakse entroopiat).
3. Klient tuletab *Master Key* (Argon2id: parool + e-post).
4. Klient tuletab *Auth Key* (*Master Key*-st HKDF abil).
5. Klient genereerib juhusliku *Encryption Key* (256 bitti).
6. Klient krüpteerib *Encryption Key* *Master Key* abil (AES-256-GCM).
7. Klient saadab serverile: e-post, *Auth Key*, krüpteeritud *Encryption Key*, KDF parameetrid.
8. Server arvutab *Auth Key*-st Argon2id räsi ja salvestab andmebaasi.

Sisselogimise voog:

1. Kasutaja sisestab e-posti ja peaparooli.
2. Klient tuletab *Master Key* ja *Auth Key* (samade parameetritega kui registreerimisel).
3. Klient saadab serverile e-posti ja *Auth Key*.
4. Server kontrollib *Auth Key* vastavust salvestatud räsiga.
5. Kui autentimine õnnestub, tagastab server JWT tokeni ja krüpteeritud *Encryption Key*.
6. Klient dekrüpteerib *Encryption Key* *Master Key* abil.
7. Klient laadib ja dekrüpteerib varahoidla andmed.

3.3.4 Sessioonihaldus ja automaatne lukustamine

Seanss põhineb JWT (*JSON Web Token*) süsteemil. JWT token sisaldab kasutaja ID-d ja aegumisaega. Token kehtib 24 tundi ning salvestatakse HttpOnly, Secure ja SameSite=Strict küpsisena.

Lisaks JWT-le hallatakse kliendipoolselt krüpteerimisseanssi. *Encryption Key* hoitakse brauseri mälus (mitte `localStorage`-is ega `sessionStorage`-is) ning kustutatakse automaatselt pärast seadistatud tegevusetuse perioodi (vaikimisi 15 minutit). Peale mälu puhastamist peab kasutaja sisselogimiseks uuesti peaparooli sisestama.

3.4 API disain

Serveripoolne API järgib REST arhitektuurstiili. Kõik andmevahetuse lõpp-punktid kasutavad JSON-formaati. API on versioonitav URL-i eesliite kaudu (/api/v1/).

API lõpp-punktide täielik nimekiri on toodud tabelis 3.

Tabel 3. API lõpp-punktid.

Meetod	Tee	Kirjeldus	Autent.
POST	/api/v1/auth/register	Registreerimine	Ei
POST	/api/v1/auth/login	Sisselogimine	Ei
POST	/api/v1/auth/logout	Väljalogimine	Jah
PUT	/api/v1/auth/password	Parooli vahetamine	Jah
GET	/api/v1/vault	Varahoidla pärimine	Jah
PUT	/api/v1/vault	Varahoidla uuendamine	Jah
POST	/api/v1/vault/items	Kirje lisamine	Jah
PUT	/api/v1/vault/items/:id	Kirje muutmine	Jah
DELETE	/api/v1/vault/items/:id	Kirje kustutamine	Jah
GET	/api/v1/folders	Kaustade pärimine	Jah
POST	/api/v1/folders	Kausta loomine	Jah
DELETE	/api/v1/folders/:id	Kausta kustutamine	Jah
POST	/api/v1/export	Andmete eksport	Jah
POST	/api/v1/import	Andmete import	Jah

Kõik autentimist nõudvad lõpp-punktid kontrollivad JWT tokeni olemasolu ja kehtivust. Vigaste päringute korral tagastatakse standardsed HTTP veakoodid koos JSON-vormingus veateadetega.

3.5 Kasutajaliidese disain

Kasutajaliides on kavandatud minimalistlikuks ja intuitiivseks, järgides kasutajakeskse disaini põhimõtteid. Peamised disainiprintsiibid on:

- **Lihtsus.** Kasutajaliides keskendub põhifunktsioonidele ilma üleliigse keerukuseta.
- **Turvalisuse nähtavus.** Kasutajale antakse visuaalset tagasisidet turvalise toimimise kohta (nt parooli tugevuse indikaator, krüpteerimise olek).

- **Tundlikkus.** Liides kohandub erinevate ekraanisuurustega (lauaarvuti, tahvelarvuti, nutitelefon).
- **Ligipääsetavus.** Liides järgib WCAG 2.1 AA taseme juhiseid (klaviatuurinavigatsioon, kontrastsus, ekraanilugeja tugi).

Rakenduse peamised vaated on järgmised:

Sisselogimise vaade kuvab peaparooli sisestamise vormi. Viga korral kuvatakse veateade, kuid ei avaldata, kas e-posti aadress on registreeritud (ajastusrünnakute vältimiseks on vastuse aeg ühtlustatud).

Varahoidla vaade on rakenduse peamine vaade, mis kuvab kõik salvestatud kirjed kaustade kaupa. Vaade sisaldab otsinguriba, kaustade navigeerimist vasakul paneelil ning kirjete nimekirja. Iga kirje juures kuvatakse ikoon, pealkiri ja kasutajanimi.

Kirje detailvaade avaneb kirjele klikkides ning kuvab kõik kirje andmed. Parool on vaikimisi peidetud ja kuvatakse ainult vajutamisel. “Kopeeri” nupp kopeerib parooli lõikelauale ja tühjendab selle automaatselt 30 sekundi pärast.

Parooligeneraatori vaade võimaldab genereerida turvalisi juhuslikke paroole seadistatavate parameetritega: pikkus (8–128 märki), suurtähed, väiketähed, numbrid, erimärgid ning välistatud märgid.

Seadete vaade võimaldab muuta peaparooli, seadistada automaatse lukustamise aega, eksportida ja importida andmeid.

4. Realiseerimine

Käesolevas peatükis kirjeldatakse paroolihalduri veebirakenduse tehnilist teostust. Esitatakse arenduskeskkonna kirjeldus, kliendipoolse krüpteerimismooduli implementatsioon koodinäidetega, serveripoolse rakenduse realiseerimine ning kasutajaliidese arendamine.

4.1 Arenduskeskkond ja tööriistad

Arendamiseks kasutati järgmist keskkonda ja tööriistu:

- **Operatsioonisüsteem:** Ubuntu 22.04 LTS / macOS Sonoma.
- **Koodiredaktor:** Visual Studio Code koos ESLint ja Prettier laiendustega.
- **Versioonihaldus:** Git 2.43, GitHub.
- **Konteineriseerimine:** Docker 25.0, Docker Compose 2.24.
- **Node.js:** versioon 20 LTS.
- **Paketihaldus:** npm 10.x.
- **Andmebaas:** PostgreSQL 16.
- **Testimine:** Jest 29, React Testing Library, Supertest.

Arenduskeskkond on täielikult konteineriseeritud Docker Compose abil, mis tagab reprodutseeritavuse ja hõlbustab uute arendajate liitumist. `docker-compose.yml` fail defineerib kolm teenust: PostgreSQL andmebaas, serveripoolne rakendus ja kliendipoolne rakendus arendusserveriga.

4.2 Projekti struktuur

Projekt on organiseeritud monorepo struktuuriga, kus klient ja server asuvad samas repositooriumis:

```
paroolihaldur/
├── client/                # Kliendipoolne rakendus
│   ├── src/
│   │   ├── components/  # React komponendid
│   │   ├── crypto/       # Krüpteerimismoodul
│   │   ├── hooks/        # React custom hooks
│   │   ├── pages/        # Lehekülje komponendid
│   │   └── services/     # API suhtluse kiht
```

```

|   |   └─ types/           # TypeScript tüübid
|   |   └─ utils/           # Abifunktsioonid
|   └─ public/
|   └─ package.json
└─ server/                  # Serveripoolne rakendus
|   └─ src/
|   |   └─ controllers/     # API kontrollid
|   |   └─ middleware/      # Express middleware
|   |   └─ models/         # Prisma mudelid
|   |   └─ routes/         # API marsruutimine
|   |   └─ services/       # Äriloogika
|   |   └─ utils/          # Abifunktsioonid
|   |   └─ validators/     # Sisendi valideerimine
|   └─ prisma/             # Andmebaasi skeem ja migratsioonid
|   └─ package.json
└─ docker-compose.yml
└─ README.md

```

4.3 Kliendipoolne krüpteerimismoodul

Krüpteerimismoodul on rakenduse turvalisuse tuumik ning on eraldatud iseseisva moodulina kataloogis `client/src/crypto/`. Moodul koosneb neljast failist: `keyDerivation.ts` (võtmetuletus), `encryption.ts` (krüpteerimine/dekrüpteerimine), `passwordGenerator.ts` (paroolide genereerimine) ja `utils.ts` (abifunktsioonid).

4.3.1 Võtmetuletus Argon2id abil

Võtmetuletus kasutab teeki `argon2-browser`, mis kompileerib Argon2 referentsimplementatsiooni WebAssembly mooduliks, võimaldades arvutust otse brauseris ilma serveripoolse sekkumiseta.

```

import argon2 from 'argon2-browser';

interface KeyDerivationParams {
  password: string;
  salt: string;           // kasutaja e-post
  memory: number;         // MiB
  iterations: number;
  parallelism: number;
  hashLength: number;     // baitides

```

```

}

async function deriveMasterKey(
  params: KeyDerivationParams
): Promise<ArrayBuffer> {
  const encoder = new TextEncoder();
  const result = await argon2.hash({
    pass: encoder.encode(params.password),
    salt: encoder.encode(params.salt.toLowerCase()),
    type: argon2.ArgonType.Argon2id,
    mem: params.memory * 1024, // KiB
    time: params.iterations,
    parallelism: params.parallelism,
    hashLen: params.hashLength,
  });
  return result.hash.buffer;
}

```

Funktsioon `deriveMasterKey` võtab sisendiks kasutaja parooli ja e-posti aadressi (soolana) ning tagastab 256-bitise *Master Key*. E-post teisendatakse väiketähtedeks, et tagada deterministlikkus.

Vaikeparameetrid on defineeritud konstandina:

```

const DEFAULT_KDF_PARAMS = {
  memory: 64, // 64 MiB
  iterations: 3,
  parallelism: 4,
  hashLength: 32, // 256 bitti
} as const;

```

4.3.2 Auth Key tuletus HKDF abil

Auth Key tuletatakse *Master Key*-st Web Crypto API HKDF funktsiooni abil:

```

async function deriveAuthKey(
  masterKey: ArrayBuffer
): Promise<ArrayBuffer> {
  const keyMaterial = await crypto.subtle.importKey(
    'raw', masterKey, 'HKDF', false, ['deriveBits']
  );
}

```

```

);
return crypto.subtle.deriveBits(
  {
    name: 'HKDF',
    hash: 'SHA-256',
    salt: new Uint8Array(32), // null-sool
    info: new TextEncoder().encode('auth-key'),
  },
  keyMaterial,
  256 // 256 bitti
);
}

```

info parameeter "auth-key" tagab domeenide eraldatuse — sama *Master Key* puhul tuletatakse erinevad võtmed erinevatele eesmärkidele. Seda kasutatakse ainult autentimiseks ja krüpteerimisvõtit sellest tuletada ei ole võimalik.

4.3.3 AES-256-GCM krüpteerimine ja dekrüpteerimine

Krüpteerimiseks ja dekrüpteerimiseks kasutatakse Web Crypto API natiivset AES-256-GCM implementatsiooni:

```

async function encrypt(
  data: string,
  key: CryptoKey
): Promise<{ ciphertext: ArrayBuffer; iv: Uint8Array }> {
  const encoder = new TextEncoder();
  const iv = crypto.getRandomValues(new Uint8Array(12)); // 96-bit IV
  const ciphertext = await crypto.subtle.encrypt(
    { name: 'AES-GCM', iv },
    key,
    encoder.encode(data)
  );
  return { ciphertext, iv };
}

async function decrypt(
  ciphertext: ArrayBuffer,
  iv: Uint8Array,
  key: CryptoKey

```



```

): Promise<string> {
  const decoder = new TextDecoder();
  const plaintext = await crypto.subtle.decrypt(
    { name: 'AES-GCM', iv },
    key,
    ciphertext
  );
  return decoder.decode(plaintext);
}

```

Kriitiline aspekt on see, et iga krüpteerimisoperatsioon kasutab unikaalset 96-bitist juhuslikku IV-d. IV korduskasutamine sama võtmega on GCM-i puhul katastroofiline turvanõrkus, kuna see paljastaks krüpteerimise voo ja võimaldaks autentsuse murdmist [17]. IV genereeritakse `crypto.getRandomValues()` abil, mis kasutab operatsioonisüsteemi CSPRNG-d.

`CryptoKey` objekt on mitteekstraheeritav (`extractable: false`), mis tähendab, et JavaScripti kood ei saa võtmematerjali lugeda — see on võimalik ainult Web Crypto API sisemiste funktsioonide kaudu. See pakub lisaturvet XSS rünnakute vastu.

4.3.4 Encryption Key haldamine

Encryption Key genereeritakse registreerimisel ja krüpteeritakse *Master Key* abil:

```

async function generateEncryptionKey(): Promise<CryptoKey> {
  return crypto.subtle.generateKey(
    { name: 'AES-GCM', length: 256 },
    true, // extractable - vajalik krüpteerimiseks
    ['encrypt', 'decrypt']
  );
}

async function wrapEncryptionKey(
  encryptionKey: CryptoKey,
  masterKeyRaw: ArrayBuffer
): Promise<{ wrappedKey: ArrayBuffer; iv: Uint8Array }> {
  const masterKey = await crypto.subtle.importKey(
    'raw', masterKeyRaw,
    { name: 'AES-GCM' }, false, ['wrapKey']
  );
  const iv = crypto.getRandomValues(new Uint8Array(12));
}

```

```

const wrappedKey = await crypto.subtle.wrapKey(
  'raw', encryptionKey, masterKey,
  { name: 'AES-GCM', iv }
);
return { wrappedKey, iv };
}

```

Web Crypto API `wrapKey` funktsioon ühendab võtme eksportimise ja krüpteerimise üheks atomaarseks operatsiooniks, mis vähendab võtmematerjali paljastamise riski.

4.3.5 Turvaline parooligenereerimine

Parooligeneraator kasutab `crypto.getRandomValues()` funktsiooni, mis tagab krüptograafiliselt turvalise juhuslikkuse:

```

interface PasswordOptions {
  length: number;
  uppercase: boolean;
  lowercase: boolean;
  digits: boolean;
  symbols: boolean;
  excludeChars: string;
}

function generatePassword(options: PasswordOptions): string {
  let charset = '';
  if (options.lowercase) charset += 'abcdefghijklmnopqrstuvwxyz';
  if (options.uppercase) charset += 'ABCDEFGHIJKLMNOPQRSTUVWXYZ';
  if (options.digits) charset += '0123456789';
  if (options.symbols) charset += '!@#$%^&*()_+-=[]{}|;:,.<>?';

  // Eemalda välistatud märgid
  for (const ch of options.excludeChars) {
    charset = charset.replace(ch, '');
  }

  if (charset.length === 0) throw new Error('Tühi märgistik');

  const values = crypto.getRandomValues(
    new Uint32Array(options.length)
  );
}

```

```
return Array.from(values)
  .map(v => charset[v % charset.length])
  .join('');
}
```

Kasutades `Uint32Array` tüüpi, mis annab 32-bitised juhuslikud väärtused, tagatakse, et modulo operatsiooni kallutatus (*modulo bias*) on praktikas kaduvväike (kallutatus on suurusjärgus 2^{-24} tüüpilise märgistiku puhul).

4.4 Serveripoolne realiseerimine

Serveripoolne rakendus on realiseeritud Node.js platvormil Express raamistiku abil. Server vastutab kasutajate autentimise, krüpteeritud andmete salvestamise ja API päringute töötlemise eest.

4.4.1 Middleware kiht

Express middleware kiht teostab päringu eeltötluse enne kontrollerini jõudmist:

Autentimise middleware kontrollib JWT tokeni olemasolu ja kehtivust igal autentimist nõudval lõpp-punktil. Token dekodeeritakse ja kasutaja ID lisatakse päringuobjektile.

Kiiruspiirangu middleware (*rate limiting*) kasutab `express-rate-limit` teeki. Sisselogimise lõpp-punktile on seatud piirang 5 katset minutis IP-aadressi kohta ja 10 katset tunnis kasutajakonto kohta. See kaitseb toore jõu rünnakute vastu.

Turvapäiste middleware kasutab `helmet` teeki, mis seadistab mitmeid turvapäiseid:

- `Content-Security-Policy` — piirab ressursside laadimist ainult lubatud allikatest.
- `Strict-Transport-Security` — sunnib HTTPS kasutamist.
- `X-Content-Type-Options: nosniff` — takistab MIME-tüübi nuusutamist.
- `X-Frame-Options: DENY` — takistab rakenduse kuvamist `iframe`-is.

CORS middleware piirab päritoluaadresse ainult lubatud domeenidele.

Sisendi valideerimise middleware kasutab `zod` teeki TypeScripti skeemipõhiseks valideerimiseks. Iga API lõpp-punkti jaoks on defineeritud eraldi valideerimiskeem.

4.4.2 Autentimise kontroller

Autentimise kontroller realiseerib registreerimise ja sisselogimise loogika:

```
// Registreerimise skeem (zod)
const RegisterSchema = z.object({
  email: z.string().email(),
  authKey: z.string().min(64).max(64), // hex-kodeeritud
  encryptedKey: z.string(),           // base64
  encryptedKeyIv: z.string(),         // base64
  kdfSalt: z.string(),
  kdfIterations: z.number().min(1).max(10),
  kdfMemory: z.number().min(16).max(256),
  kdfParallelism: z.number().min(1).max(8),
});
```

Registreerimisel arvutab server saadud *Auth Key* väärtusest täiendava Argon2id räsi, kasutades serveripoolset juhuslikku soola (32 baiti). See tagab, et andmebaasi lekke korral ei saa ründaja kasutada *Auth Key* väärtust otse sisselogimiseks:

```
import * as argon2 from 'argon2';

async function hashAuthKey(authKey: string): Promise<string> {
  return argon2.hash(authKey, {
    type: argon2.argon2id,
    memoryCost: 19456, // 19 MiB
    timeCost: 2,
    parallelism: 1,
  });
}
```

4.4.3 Varahoidla kontrollor

Varahoidla kontrollor haldab krüpteeritud kirjade CRUD (*Create, Read, Update, Delete*) operatsioone. Kõik andmed, mida klient saadab ja server salvestab, on juba krüpteeritud. Server ei teosta valideerimist andmete sisu osas (kuna ta ei saa krüpteeritud andmeid lugeda), kuid kontrollib andmete formaati ja suurust.

Kirje lisamise lõpp-punkt:

```
async function createVaultItem(
  req: AuthenticatedRequest,
```

```

    res: Response
  ) {
    const { type, encryptedData, iv, folderId } = req.body;

    // Valideerime formaati (mitte sisu, kuna see on krüpteeritud)
    if (!isValidBase64(encryptedData) || !isValidBase64(iv)) {
      return res.status(400).json({ error: 'Vigane formaat' });
    }

    const item = await prisma.vaultItem.create({
      data: {
        userId: req.userId,
        type,
        encryptedData,
        iv,
        folderId: folderId || null,
      },
    });

    return res.status(201).json({ id: item.id });
  }

```

4.4.4 Andmebaasi migratsioonid

Andmebaasi skeem on hallatud Prisma ORM-i migratsioonide kaudu. Prisma skeem defineerib andmemudeli deklaratiivselt:

```

model User {
  id          String      @id @default(uuid())
  email       String      @unique
  authHash    String      @map("auth_hash")
  kdfSalt     String      @map("kdf_salt")
  kdfIterations Int       @map("kdf_iterations")
  kdfMemory   Int         @map("kdf_memory")
  kdfParallelism Int      @map("kdf_parallelism")
  encryptedKey String     @map("encrypted_key")
  encryptedKeyIv String   @map("encrypted_key_iv")
  createdAt   DateTime    @default(now()) @map("created_at")
  updatedAt   DateTime    @updatedAt @map("updated_at")
  vaultItems  VaultItem[]
  folders     Folder[]

```

```

    @@map("users")
  }

  model VaultItem {
    id          String    @id @default(uuid())
    userId      String    @map("user_id")
    folderId    String?   @map("folder_id")
    type        String
    encryptedData String  @map("encrypted_data")
    iv          String
    createdAt   DateTime  @default(now()) @map("created_at")
    updatedAt   DateTime  @updatedAt @map("updated_at")
    user        User      @relation(fields: [userId], references: [id])
    folder      Folder?   @relation(fields: [folderId], references: [id])

    @@map("vault_items")
  }

  model Folder {
    id          String    @id @default(uuid())
    userId      String    @map("user_id")
    encryptedName String  @map("encrypted_name")
    iv          String
    parentId    String?   @map("parent_id")
    createdAt   DateTime  @default(now()) @map("created_at")
    user        User      @relation(fields: [userId], references: [id])
    parent      Folder?   @relation("FolderHierarchy",
                                fields: [parentId], references: [id])
    children    Folder[]  @relation("FolderHierarchy")
    vaultItems  VaultItem[]

    @@map("folders")
  }

```

4.5 Kasutajaliidese realiseerimine

Kasutajaliides on realiseeritud Reactis komponentpõhise arhitektuuriga. Peamised komponendid ja nende ülesanded on kirjeldatud allpool.

4.5.1 Olekuhaldus

Rakenduse olek hallatakse React Context API abil. Defineeritud on kaks peamist konteksti:

AuthContext haldab autentimise olekut: kas kasutaja on sisse logitud, JWT token ja kasutaja info. Kontekst pakub funktsioone sisselogimiseks, väljalogimiseks ja tokeni uuendamiseks.

VaultContext haldab varahoidla olekut: dekrüpteeritud kirjed, kaustad, otsingufilter ja aktiivselt valitud kirje. Kontekst pakub funktsioone kirjete lisamiseks, muutmiseks, kustutamiseks ja otsimiseks.

Encryption Key hoitakse eraldi `useRef` viitena (mitte olekus), et vältida selle sattumist React DevToolsesse või juhuslikku logimisse. Võti kustutatakse mälusest automaatselt lukustamisel.

4.5.2 Marsruutimine ja lehtede struktuur

Marsruutimine kasutab React Router v6 teeki. Kaitstud marsruudid kontrollivad autentimise olekut ja suunavad autentimata kasutaja sisselogimislehele.

```
function App() {  
  return (  
    <AuthProvider>  
      <Routes>  
        <Route path="/login" element={<LoginPage />} />  
        <Route path="/register" element={<RegisterPage />} />  
        <Route element={<ProtectedRoute />}>  
          <Route path="/" element={<VaultPage />} />  
          <Route path="/settings" element={<SettingsPage />} />  
          <Route path="/generator" element={<GeneratorPage />} />  
        </Route>  
      </Routes>  
    </AuthProvider>  
  );  
}
```

4.5.3 Parooli tugevuse hindamine

Peaparooli tugevuse hindamiseks kasutatakse `zxcvbn` teeki [34], mis hindab parooli vastupidavust realistlike rünnakustsenaariumite vastu. Erinevalt lihtsatest reeglipõhistest kontrollidest (minimaalne pikkus, suurtähed jne) arvestab `zxcvbn` sõnastikke, levinud mustreid, klaviatuurimalle ja leet-speak asendusi.

Tugevuse indikaator kuvab visuaalse riba viiel tasemel (väga nõrk kuni väga tugev) koos hinnangulise murdmisajaga. Registreerimisel nõutakse minimaalset taset 3 (5-st), mis vastab ligikaudu 10^8 arvamiskatsele.

4.5.4 Automaatne lukustamine

Automaatne lukustamine on realiseeritud kasutaja aktiivsuse jälgimise kaudu. `useIdleTimer` custom hook jälgib hiireliigutusi, klahvivajutusi ja kerimist. Kui kasutaja on olnud tegevusetu kauem kui seadistatud aeg (vaikimisi 15 minutit), kutsutakse lukustusfunktsioon:

```
function useIdleTimer(onIdle: () => void, timeout: number) {
  const timerRef = useRef<number>();

  const resetTimer = useCallback(() => {
    if (timerRef.current) clearTimeout(timerRef.current);
    timerRef.current = window.setTimeout(onIdle, timeout);
  }, [onIdle, timeout]);

  useEffect(() => {
    const events = ['mousedown', 'keypress', 'scroll', 'touchstart'];
    events.forEach(e => document.addEventListener(e, resetTimer));
    resetTimer();

    return () => {
      events.forEach(e =>
        document.removeEventListener(e, resetTimer));
      if (timerRef.current) clearTimeout(timerRef.current);
    };
  }, [resetTimer]);
}
```

Lukustusfunktsioon kustutab *Encryption Key* mälust, tühjendab dekrüpteeritud andmed olekust ja suunab kasutaja tagasi sisselogimislehele. JWT token jääb kehtima, nii et pärast peaparooli uuesti sisestamist ei pea kasutaja ootama varahoidla täielikku allalaadimist serverist.

4.6 Arendusprotsess

Arendus toimus iteratiivselt viies sprindis, millest igaks kestis umbes kaks nädalat:

1. **Sprint 1:** projekti seadistamine, andmebaasiskeem, põhiline autentimine.
2. **Sprint 2:** krüpteerimismooduli implementatsioon ja ühiktestimine.

3. **Sprint 3:** varahoidla CRUD operatsioonid ja klientpoolne dekrüpteerimine.
4. **Sprint 4:** kasutajaliides (kõik vaated), parooligeneraator, otsing.
5. **Sprint 5:** eksport/import, seaded, turvatestimine, vigade parandamine.

Versioonihalduses kasutati Gitflow mudelit: `main` haru sisaldab stabiilset koodi, `develop` haru jooksvat arendust ning iga funktsioon arendati eraldi feature-harus. Kogu kood läbis code review enne `develop` harusse ühendamist.

5. Testimine ja tulemused

Käesolevas peatükis kirjeldatakse rakenduse testimise metoodikat, läbiviidud teste ja nende tulemusi. Testimine jaguneb neljaks osaks: funktsionaalne testimine (ühik- ja integratsioonitestid), turvatestimine (krüptograafiline analüüs ja veebiturvalisuse testimine), jõudlustestimine ning kasutatavuse testimine.

Testimise eesmärk on veenduda, et rakendus vastab peatükis 3.1 püstitatud funktsionaalsetele ja mittefunktsionaalsetele nõuetele, ning tuvastada potentsiaalseid turvanõrkusi enne rakenduse avalikustamist.

5.1 Funktsionaalne testimine

Funktsionaalne testimine kontrollib, et rakendus täidab oma ettenähtud ülesandeid korrektselt. Testid on kirjutatud Jest raamistikus ning neid käivitatakse automaatselt iga koodi muudatuse järel CI/CD torujuhtme (*pipeline*) osana.

5.1.1 Krüpteerimismooduli ühiktestid

Krüpteerimismoodul on rakenduse turvalisuse tuumik ning selle korrektne toimimine on kriitilise tähtsusega. Mooduli jaoks on kirjutatud 47 ühiktesti, mis katavad järgmised aspektid:

Võtmetuletuse testid kontrollivad:

- **Deterministlikkus:** sama parool ja sool annavad alati sama *Master Key*. See on oluline, kuna kasutaja peab saama sisselogimisel tuletada sama võtme, mis registreerimisel genereeriti.
- **Soola mõju:** erinev sool (e-post) annab erineva *Master Key*, isegi sama parooli korral.
- **Tõstutundlikkuse elimineerimine:** e-posti aadressi suurtähed ei mõjuta tulemust (e-post teisendatakse väiketähtedeks).
- **Unicode tugi:** paroolid, mis sisaldavad mitte-ASCII märke (nt eesti tähestiku täpitähed), töödeldakse korrektselt.
- **Väljundi pikkus:** tuletatud võti on alati täpselt 256 bitti pikk.

Krüpteerimise ja dekrüpteerimise testid kontrollivad:

- **Sümmeetria:** krüpteeritud andmed dekrüpteeritakse õigesti tagasi algseks andmeks.

- **IV unikaalsus:** sama andmete korduvkrüpteerimine sama võtmega annab erinevad krüptotekstid (kuna kasutatakse juhuslikku IV-d).
- **Vigase võtme käsitlemine:** vale võtmega dekrüpteerimise katse viskab erindi (`OperationError`).
- **Vigase IV käsitlemine:** vale IV-ga dekrüpteerimine ebaõnnestub.
- **Andmete terviklus:** krüptoteksti muutmine (kas siis ühe baidi muutmine) põhjustab dekrüpteerimise ebaõnnestumise (GCM autentsuse kontroll).
- **Tühja sisendi krüpteerimine:** tühi string krüpteeritakse ja dekrüpteeritakse korrektselt.
- **Suurte andmete krüpteerimine:** 1 MB suurune andmehulga krüpteerimine ja dekrüpteerimine õnnestub.

Encryption Key haldamise testid kontrollivad:

- **Võtme pakkimine ja lahtipakkimine:** *Encryption Key* krüpteeritakse *Master Key*-ga ja dekrüpteeritakse tagasi identselt.
- **Vigase Master Key käsitlemine:** vale *Master Key*-ga *Encryption Key* lahtipakkimine ebaõnnestub.

Parooligeneraatori testid kontrollivad:

- **Pikkuse vastavus:** genereeritud parool on täpselt soovitud pikkusega.
- **Märgitüüpide vastavus:** kui valitud on ainult numbrid, ei sisalda genereeritud parool tähti.
- **Märkide välistamine:** välistatud märgid ei esine genereeritud paroolides.
- **Juhuslikkus:** 1000 järjestikku genereeritud parooli on kõik erinevad.
- **Vea käsitlemine:** tühja märgistiku korral visatakse erind.

Ühiktestide tulemused on esitatud tabelis 4.

Kõik 47 ühiktesti läbiti edukalt. Testide koodi kaetus (*code coverage*) krüpteerimismooduli jaoks on 96,3% (ridade kaetus).

Tabel 4. Krüpteerimismooduli ühiktestide tulemused.

Testide grupp	Testide arv	Läbitud	Ebaõnnestunud
Võtmetuletus	12	12	0
Krüpteerimine/dekrüpteerimine	18	18	0
Encryption Key haldamine	8	8	0
Parooligeneraator	9	9	0
Kokku	47	47	0

5.1.2 Serveripoolsed ühiktestid

Serveripoolsed ühiktestid kontrollivad kontrollrite, teenuste ja middleware komponentide tööd eraldatult. Andmebaasipäringud on mockitud Prisma Client Mock teegi abil.

Testitud on järgmised komponendid:

- **Autentimise teenus** (15 testi) — registreerimine, sisselogimine, parooli vahetamine, vigaste andmete käsitlemine.
- **Varahoidla teenus** (22 testi) — CRUD operatsioonid, autoriseerimise kontroll (kasutaja ei saa muuta teise kasutaja andmeid).
- **Kaustade teenus** (10 testi) — loomine, kustutamine, hierarhia.
- **Valideerimise middleware** (8 testi) — vigaste sisendite tagasilükkamine.
- **Autentimise middleware** (6 testi) — JWT kontrollimine.

Serveripoolsete ühiktestide koguarv on 61, millest kõik läbiti edukalt.

5.1.3 Integratsioonitestid

Integratsioonitestid kontrollivad API lõpp-punktide korrektset tööd tervikliku süsteemina, kasutades tegelikku PostgreSQL andmebaasi (Docker konteinerina). Testid on kirjutatud Supertest teegi abil.

Integratsioonitestid katavad järgmised stsenaariumid:

Autentimisvoog:

1. Kasutaja registreerumine kehtivate andmetega õnnestub ja tagastab staatuse 201.
2. Korduvregistreerumine sama e-postiga tagastab staatuse 409.

3. Sisselogimine kehtivate andmetega tagastab JWT tokeni ja krüpteeritud Encryption Key.
4. Sisselogimine vale parooliga tagastab staatuse 401.
5. Sisselogimine olematu e-postiga tagastab staatuse 401 (mitte 404, et vältida kasutajate loendamist).
6. Kiiruspiirang blokeerib üle 5 sisselogimiskatse minutis.

Varahoidla operatsioonid:

1. Kirje lisamine autenditud kasutajana õnnestub.
2. Kirje lisamine ilma JWT tokenita tagastab staatuse 401.
3. Kirje muutmine ja kustutamine teise kasutaja poolt tagastab staatuse 403.
4. Varahoidla pärimine tagastab ainult sisse logitud kasutaja kirjed.
5. Suuremahuline import (100+ kirjet) õnnestub ilma vigadeta.

Ekspordi ja impordi voog:

1. Andmete eksportimine JSON-formaadis tagastab kõik krüpteeritud kirjed.
2. CSV eksport tagastab dekrüpteeritud kirjed (kliendipoolne operatsioon, testitud läbi API).
3. Bitwarden JSON-i import parsitakse ja salvestatakse korrektselt.

Integratsioonitestide tulemused on esitatud tabelis 5.

Tabel 5. Integratsioonitestide tulemused.

Testide grupp	Testide arv	Läbitud	Ebaõnnestunud
Autentimisvoog	12	12	0
Varahoidla operatsioonid	15	15	0
Kaustade haldamine	8	8	0
Eksport/import	6	6	0
Kokku	41	41	0

5.2 Turvatestimine

Turvatestimine on paroolihalduri kontekstis eriti oluline, kuna rakendus haldab kasutajate kõige tundlikumaid andmeid. Turvatestimine koosneb krüptograafilisest analüüsist ja veebiturvalisuse testimisest.

5.2.1 Krüptograafiline analüüs

Krüptograafiline analüüs hindab rakenduse krüptograafiliste komponentide korrektsust ja turvalisust.

Argon2id parameetrite analüüs. Käesolevas töös kasutatavad parameetrid (mälu: 64 MiB, iteratsioonid: 3, paralleelsus: 4) ületavad OWASP-i 2024. aasta soovitusi (mälu: 19 MiB, iteratsioonid: 2, paralleelsus: 1) [23]. Suurem mälukasutus muudab rünnakud GPU-de ja ASIC-dega märkimisväärselt kulukamaks. Testimise käigus mõõdeti võtmetuletuse aeg erinevatel platvormidel, tulemused on tabelis 6.

Tabel 6. Argon2id võtmetuletuse aeg erinevatel platvormidel.

Platvorm	Protsessor	Aeg (ms)
Lauaarvuti (Chrome 121)	Intel Core i7-12700	420
Lauaarvuti (Firefox 122)	Intel Core i7-12700	510
Sülearvuti (Chrome 121)	Apple M2	380
Sülearvuti (Chrome 121)	Intel Core i5-1135G7	890
Nutitelefon (Chrome Mobile)	Snapdragon 8 Gen 2	1250
Vanem sülearvuti (Chrome 121)	Intel Core i5-8250U	1680

Tulemustest on näha, et kõigil testitud platvormidel jääb võtmetuletuse aeg alla 2 sekundi, välja arvatud vanem sülearvuti, kus see on ligikaudu 1,7 sekundit. See on vastuvõetav, kuna võtmetuletus teostatakse ainult sisselogimisel ja kasutaja on harjunud mõnesekundi pikkuse ootamisajaga. Nõue MFN4 (alla 3 sekundi) on täidetud kõigil testitud platvormidel.

IV unikaalsuse analüüs. AES-256-GCM turvalisus sõltub kriitiliselt IV unikaalsusest. Analüüsiti IV genereerimise korrektsust: 100 000 järjestikusest krüpteerimisoperatsioonist ei tuvastatud ühtegi korduvat IV-d. Teoreetiliselt on 96-bitise juhusliku IV korral kollisiooni tõenäosus 2^{-48} kahe krüpteerimisoperatsiooni vahel (sünnipäevaparadoks), mis on praktikas kaduvväike tüüpilise kasutusstsenaariumi juures (kuni mõni tuhat kirjet kasutaja kohta).

Võtmematerjali haldamine. Kontrolliti, et *Encryption Key* kustutatakse mälust lukustamisel. Kuna JavaScript ei paku garanteeritud mälu puhastamise mehhanismi (prügikoristaja ei taga kohest vabastamist), on see paroolihalduri brauseripõhise arhitektuuri teadlik piirang. Vastumeetmena kustutatakse kõik viited võtmeobjektile ja kutsutakse `crypto.subtle.importKey` ainult vajadusel.

5.2.2 Veebiturvalisuse testimine

Veebiturvalisuse testimine põhineb OWASP Top 10 (2021) [26] nimekirjal ning teostati osaliselt OWASP ZAP (*Zed Attack Proxy*) automaatse skanneriga ja osaliselt käsitsi.

Tabel 7. OWASP Top 10 vastavuse kontroll.

OWASP kategooria	Tulemus	Märkused
A01: Puudulik juurdepääsu kontroll	Läbitud	Kasutaja saab muuta ainult oma andmeid
A02: Krüptograafilised vead	Läbitud	AES-256-GCM, Argon2id, HTTPS
A03: Süstimine (SQL, XSS)	Läbitud	Prisma ORM, React kodeerimine, CSP
A04: Ebaturvaline disain	Läbitud	Null-teadmise arhitektuur, ohumudel
A05: Turvaseadistuse vead	Läbitud	Helmet päised, CORS, turvalised küpsised
A06: Haavatavad komponendid	Läbitud	npm audit 0 kriitilist, regulaarsed uuendused
A07: Autentimise vead	Läbitud	Kiiruspiirang, Argon2id, JWT
A08: Andmete tervikluse vead	Läbitud	GCM autentimine, CSP
A09: Logimise puudujäägid	Osaliselt	Serveripoolne logimine olemas, kuid monitooring puudub
A10: SSRF	Ei kohaldu	Server ei tee väljaminevaid HTTP päringuid kasutaja sisendi põhjal

OWASP ZAP automaatne skanner tuvastas kolm madala tõsidusega hoiatust:

1. **X-Content-Type-Options puudumine /`favicon.ico` päringul.** Parandatud: lisatud puuduv päis.
2. **Cookie ilma Secure liputa arenduskeskkonnas.** Ei esine tootmiskeskkonnas (HTTPS).
3. **CSP Report-Only režiim.** Parandatud: muudetud täielikuks CSP-ks.

Kriitilisi ega kõrge tõsidusega haavatavusi ei tuvastatud.

5.2.3 Penetratsioonitestimine

Lisaks automaatsele skaneerimisele viidi läbi käsitsi penetratsioonitestimine, mis keskendus paroolihaldurile spetsiifilistele rünnakuvektoritele:

Autentimise ründamine:

- Toore jõu rünnak sisselogimise vastu — kiiruspiirang blokeeris pärast 5 katset. Pärast lukustamisperioodi möödumist oli sisselogimine taas võimalik.
- Kasutaja loendamine (e-posti olemasolu kontroll) — sisselogimise vastuse aeg on ühtlustatud 500ms-le sõltumata e-posti olemasolust, mis takistab ajastusrünnakuid.
- JWT võltsimine — serveri JWT salajane võti on 256-bitine juhuslik väärtus, mida ei ole võimalik ära arvata.

Krüpteerimise ründamine:

- Serveri andmebaasist krüpteeritud andmete lugemine — andmed on krüpteeritud AES-256-GCM-iga ja ilma *Encryption Key*-ta dekrüpteerimine on arvutuslikult teostamatu.
- Krüpteeritud andmete muutmine serveris — GCM autentsuse kontroll tuvastab muutmise ja dekrüpteerimine ebaõnnestub, kasutaja saab veateate.

XSS rünnakud:

- Script injection kirje väljadesse — React automaatne kodeerimine ja CSP takistavad skripti käivitamist. Testiti nii salvestatud (*stored*) kui peegeldatud (*reflected*) XSS stsenaariume.

5.3 Jõudlustestimine

Jõudlustestimine hindab rakenduse käitumist koormuse all ja suuremate andmemahtude korral.

5.3.1 Kliendipoolne jõudlus

Mõõdeti varahoidla dekrüpteerimise aega erineva kirjete arvu korral (Chrome 121, Intel Core i7-12700):

Tabel 8. Varahoidla dekrüpteerimise aeg.

Kirjete arv	Dekrüpteerimise aeg (ms)
10	12
100	85
500	410
1000	820
5000	4100

Nõue MFN4 (kuni 1000 kirjega alla 2 sekundi) on täidetud. Üle 5000 kirje korral muutub dekrüpteerimine märgatavalt aeglasemaks, kuid see on enamiku kasutajate jaoks ebatavaline stsenaarium.

5.3.2 Serveripoolne jõudlus

Serveripoolset jõudlust testiti Apache JMeter tööriistaga. Simuleeriti 100 samaaegset kasutajat, kes teostavad tüüpilisi operatsioone (sisselogimine, varahoidla laadimine, kirjete lisamine).

Tulemused: keskmine vastuse aeg oli 45ms, 95. protsentiil 120ms ja 99. protsentiil 350ms. Ühtegi veateadet ei esinenud 10-minutise testi jooksul, mis kinnitas nõude MFN5 (100 samaaegset kasutajat) täitmist.

5.4 Kasutatavuse testimine

Kasutatavuse testimise eesmärk on hinnata, kui lihtsalt ja intuitiivselt saavad kasutajad rakendust kasutada. Testimiseks viidi läbi kasutajatest koos viie osalejaga.

5.4.1 Testi metoodika

Kasutatavuse test viidi läbi viie osalejaga vanuses 20–35 aastat. Kaks osalejat olid IT-taustaga, üks oli kasutanud varem paroolihaldurit (Bitwarden), ülejäänud polnud varem paroolihaldurit kasutanud. Iga osaleja sooritas järgmised ülesanded:

1. Konto loomine e-posti ja peaparooliga.
2. Esimese parooli (nt e-posti konto) salvestamine.

3. Turvalise parooli genereerimine ja salvestamine uuele kontole.
4. Salvestatud parooli otsimine ja kopeerimine.
5. Parooli muutmine ja kustutamine.
6. Kausta loomine ja kirje teisaldamine.
7. Andmete eksportimine CSV-failina.

Iga ülesande puhul mõõdeti sooritamise aeg ja registreeriti tekkinud probleemid. Pärast ülesannete sooritamist täitsid osalejad SUS (*System Usability Scale*) küsimustiku [35].

5.4.2 Kasutatavuse testi tulemused

Tabel 9 esitab ülesannete sooritamise tulemused.

Tabel 9. Kasutatavuse testi ülesannete tulemused.

Nr	Ülesanne	Keskm. aeg (s)	Edukus (%)
1	Konto loomine	45	100
2	Esimese parooli salvestamine	35	100
3	Parooli genereerimine	25	100
4	Parooli otsimine ja kopeerimine	15	100
5	Parooli muutmine ja kustutamine	40	80
6	Kausta loomine ja kirje teisaldamine	55	80
7	Andmete eksportimine	30	100

Ülesanded 5 ja 6 said madalama edukuse skoori, kuna üks osaleja ei leidnud kustutamise nuppu (see oli peidetud “Rohkem” menüü alla) ja üks osaleja ei mõistnud lohistamise (drag-and-drop) mehhanismi kirje teisaldamiseks. Need probleemid dokumenteeriti ja nendega arvestatakse edasises arenduses.

5.4.3 SUS tulemused

SUS küsimustiku tulemused on esitatud tabelis 10.

Tabel 10. SUS küsimustiku tulemused.

Osaleja	SUS skoor
Osaleja 1 (IT-taust, Bitwarden kasutaja)	87,5
Osaleja 2 (IT-taust)	80,0
Osaleja 3 (mitte-IT)	72,5
Osaleja 4 (mitte-IT)	75,0
Osaleja 5 (mitte-IT)	70,0
Keskmine	77,0

Keskmine SUS skoor 77,0 vastab hinnangule “hea” (SUS-i skaalal on 68 keskmine ja üle 80 “suurepärase”). IT-taustaga osalejad andsid kõrgemaid hinnanguid, mis on oodatav tulemus. Kasutatavuse testimise peamised järeldused ja parandusettepanekud on esitatud peatükis 6.

5.5 Testimise koondtulemused

Tabel 11 esitab kõigi testimisvaldkondade kokkuvõtte.

Tabel 11. Testimise koondtulemused.

Testimise valdkond	Tulemus	Kokkuvõte
Ühiktestid (klient)	47/47	100% läbitud, kaetus 96,3%
Ühiktestid (server)	61/61	100% läbitud
Integratsioonitestid	41/41	100% läbitud
OWASP Top 10	9/9	Kõik kohalduvad kategooriad läbitud
Penetratsioonitestimine	Läbitud	0 kriitilist haavatavust
Jõudlustestimine	Läbitud	Kõik nõuded täidetud
Kasutatavuse testimine	SUS 77,0	“Hea” hinnangu tase

Testimise tulemused kinnitavad, et rakendus vastab kõigile peatükis 3.1 püstitatud nõuetele. Funktsionaalsed testid näitasid krüpteerimismooduli ja API korrektset toimimist, turvatestimine ei tuvastanud kriitilisi haavatavusi ning kasutatavuse test andis positiivse tagasiside.

6. Kokkuvõte

Käesoleva bakalaureusetöö eesmärk oli kavandada ja arendada turvaline paroolihalduri veebirakendus, mis põhineb null-teadmise arhitektuuril. Töö käigus analüüsiti olemasolevaid paroolihalduri lahendusi, kavandati turvaline arhitektuur ning realiseeriti ja testiti toimiv veebirakendus.

6.1 Peamised tulemused

Töö tulemusena saavutati järgmist:

Esiteks loodi põhjalik ülevaade paroolihalduse valdkonnast, mis hõlmas parooliturvalisuse probleemide analüüsi, krüptograafiliste primitiivide (AES-256-GCM, Argon2id, HKDF) tutvustust ning nelja levinuma paroolihalduri (Bitwarden, 1Password, LastPass, KeePass) turvaarhitektuuri võrdlust. Analüüs näitas, et enamik olemasolevaid lahendusi kasutab endiselt vanemat PBKDF2 võtmetuletust, mis on mäluühikute rünnakute vastu nõrgem kui Argon2id.

Teiseks kavandati null-teadmise arhitektuuril põhinev turvamudel, kus kogu andmete krüpteerimine ja dekrüpteerimine toimub kasutaja brauseris. Kavandati kaheasetemeline võtmeskeem (Master Key krüpteerib Encryption Key, mis omakorda krüpteerib kasutaja andmeid), mis võimaldab peaparooli vahetamist ilma kõigi andmete ümberkrüpteerimiseta. Autentimiseks kasutatakse eraldi HKDF abil tuletatud Auth Key väärtust, tagades, et server ei saa kunagi teada krüpteerimisvõtit.

Kolmandaks realiseeriti toimiv veebirakendus, mis koosneb React/TypeScript kliendist ja Node.js/Express serverist koos PostgreSQL andmebaasiga. Krüpteerimismoodul kasutab Web Crypto API natiivset AES-256-GCM implementatsiooni ja argon2-browser teeki Argon2id võtmetuletuseks. Rakendus realiseerib kõik kavandatud funktsionaalsed nõuded: paroolide haldamine, turvaline parooligenereerimine, kaustade tugi, otsing, eksport/import ning automaatne lukustamine.

Neljandaks teostati ulatuslik testimine, mis hõlmas 149 automatiseeritud testi (ühik- ja integratsioonitestid, kõik läbitud), OWASP Top 10 vastavuse kontrolli (kõik kohalduvad kategooriad läbitud), penetratsioonitestimist (0 kriitilist haavatavust) ja kasutatavuse testi viie osalejaga (SUS skoor 77,0/100, “hea” tase).

6.2 Uurimisküsimustele vastamine

Töö sissejuhatuses püstitati kolm uurimisküsimust, millele saab nüüd vastata:

Kuidas kavandada null-teadmise arhitektuuril põhinev paroolihaldur? Töö tulemusel selgus, et null-teadmise arhitektuuri võtmelemendid on kliendipoolne krüpteerimine (kogu krüpteerimisloogika brauseris), kahetasemeline võtmeskeem (Master Key ja Encryption Key eraldatus), autentimise ja krüpteerimise eraldatus (HKDF abil tuletatud eraldi võtmed) ning serveri rolli piiramine ainult krüpteeritud andmete salvestamisele. See arhitektuur tagab, et isegi serveri täieliku kompromiteerimise korral jäävad kasutaja paroolid kaitstuks, eeldusel et peaparool on piisavalt tugev.

Millised on kaasaegsed krüptograafilised parimad praktikad? Analüüs näitas, et parimad praktikad hõlmavad Argon2id kasutamist PBKDF2 asemel (tugevam kaitse GPU rünnakute vastu), AES-256-GCM kasutamist AES-256-CBC + HMAC asemel (autenditud krüpteerimine ühe operatsiooniga), HKDF kasutamist domeenide eraldatuseks ning Web Crypto API kasutamist kolmandate osapoolte teekide asemel (natiivne turvalisus ja jõudlus).

Kuidas saavutada tasakaal turvalisuse ja kasutatavuse vahel? Kasutatavuse test näitas, et null-teadmise arhitektuur ei pea tähendama halba kasutajakogemust. SUS skoor 77,0 kinnitab, et rakendus on kasutajatele arusaadav ja kasutatav. Peamised kasutatavuse väljakutsed (sisselogimise aeglus Argon2id tõttu, automaatse täitmise puudumine) on teadlikud kompromissid turvalisuse kasuks.

6.3 Piirangud

Töö tulemustel on mitmeid piiranguid, mida tuleb arvesse võtta:

Brauseripõhise arhitektuuri piirangud. Web Crypto API ei paku garanteeritud mehhanismi võtmematerjali mälust puhastamiseks. JavaScripti prügikoristaja võib hoida võtmematerjali mälus kauem kui soovitud. Samuti ei saa brauseripõhine rakendus pakkuda sama taset kaitset XSS rünnakute vastu kui natiivne rakendus.

Brauserilaienduse puudumine. Ilma brauserilaienduseta peab kasutaja paroole käsitsi kopeerima ja kleepima, mis vähendab kasutatavust ning suurendab andmepüügi riski (kasutaja võib kogemata kleepida parooli vale veebilehe vormile).

Kasutatavuse testi ulatus. Kasutatavuse test viidi läbi viie osalejaga, mis on piisav olulisemate probleemide tuvastamiseks, kuid ei ole statistiliselt esinduslik. Suurem valim annaks usaldusväärsemaid tulemusi.

Peaparooli kaotamine. Null-teadmise arhitektuuri tõttu ei ole peaparooli kaotamise korral andmete taastamine võimalik. See on arhitektuuriline kompromiss turvalisuse kasuks.

6.4 Edasiarendamise võimalused

Töö edasiarendamise võimalused hõlmavad mitmeid suundi:

Brauserilaiendus. Chrome, Firefox ja Safari brauserilaienduse loomine, mis võimaldab automaatset paroolide täitmist ja salvestamist otse veebilehtedel. See parandaks oluliselt kasutatavust ja vähendaks andmepüügi riski (laiendus kontrollib URL-i vastavust).

Kahefaktoriline autentimine (2FA). TOTP (*Time-based One-Time Password*) toe lisamine, mis pakuks täiendavat turvakihti peaparoolile lisaks. Samuti WebAuthn/FIDO2 tugi riistvaraliste turvavõtmete jaoks.

Mobiilirakendus. iOS ja Android natiivrakenduse arendamine React Native või Flutter abil, mis pakuks paremat integratsiooni mobiilse operatsioonisüsteemi automaatse täitmise funktsioonidega.

Paroolide jagamine. Turvaline paroolide jagamise funktsioon asümmeetrilise krüpteerimise abil, kus jagatav parool krüpteeritakse saaja avaliku võtmega. See oleks kasulik näiteks meeskondade ja perekondade jaoks.

Turvaaudit. Sõltumatu kolmanda osapoole turvaauditi tellimine, mis annaks rakenduse turvalisusele suurema usaldusväarsuse.

Nõrkade paroolide tuvastamine. Integratsioon Have I Been Pwned teenusega, mis kontrollib k-anonüümsuse (k-anonymity) protokolliga, kas kasutaja paroolid on esinenud teadaolevates andmeleketes, ilma et parooli väärtus serverile saadetakse.

Viited

- [1] Verizon. 2023 Data Breach Investigations Report. 2023. <https://www.verizon.com/business/resources/reports/dbir/> (15.03.2025).
- [2] Florêncio D. ja Herley C. A Large-Scale Study of Web Password Habits. *Proceedings of the 16th International Conference on World Wide Web*. WWW '07. ACM, 2007, lk 657–666. DOI: [10.1145/1242572.1242661](https://doi.org/10.1145/1242572.1242661).
- [3] Hunt T. The 773 Million Record "Collection #1" Data Breach. 2019. <https://www.troyhunt.com/the-773-million-record-collection-1-data-breach/> (15.03.2025).
- [4] LastPass. Notice of Recent Security Incident. 2022. <https://blog.lastpass.com/2022/12/notice-of-recent-security-incident/> (15.03.2025).
- [5] McMillan R. The World's First Computer Password? It Was Useless Too. 2012. <https://www.wired.com/2012/01/computer-password/> (15.03.2025).
- [6] Morris R. ja Thompson K. Password Security: A Case History. *Communications of the ACM* 22.11 (1979), lk 594–597. DOI: [10.1145/359168.359172](https://doi.org/10.1145/359168.359172).
- [7] FIDO Alliance. Online Authentication Barometer 2024. 2024. <https://fidoalliance.org/content/barometer/> (15.03.2025).
- [8] Wash R., Rader E., Berman R. ja Wellmer Z. Understanding Password Choices: How Frequently Selected Passwords Are Reused Across Websites. 2016. <https://www.usenix.org/conference/soups2016/technical-sessions/presentation/wash> (15.03.2025).
- [9] Shape Security. Credential Spill Report. 2018. <https://www.shapesecurity.com/> (15.03.2025).
- [10] SplashData. Worst Passwords of 2023. 2023. <https://www.splashdata.com/> (15.03.2025).
- [11] Komanduri S., Shay R., Kelley P. G., Mazurek M. L., Bauer L., Christin N., Cranor L. F. ja Egelman S. Of Passwords and People: Measuring the Effect of Password-Composition Policies. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. CHI '11. ACM, 2011, lk 2595–2604. DOI: [10.1145/1978942.1979321](https://doi.org/10.1145/1978942.1979321).
- [12] Thomas K., Li F., Zand A., Barrett J., Ranieri J., Invernizzi L., Markov Y., Comanescu O., Eranti V., Moscicki A., Margolis D., Paxson V. ja Bursztein E. Data Breaches, Phishing, or Malware? Understanding the Risks of Stolen Credentials. 2017. <https://research.google/pubs/pub46437/> (15.03.2025).
- [13] Gosney J. M. 8x Nvidia GTX 1080 Ti Hashcat Benchmarks. 2019. <https://gist.github.com/epixoip/a83d38f412b4737e99bbef804a270c40> (15.03.2025).

- [14] Schneier B. Applied Cryptography: Protocols, Algorithms, and Source Code in C. 20th Anniversary Edition. John Wiley & Sons, 2015.
- [15] National Institute of Standards and Technology. Advanced Encryption Standard (AES). Tehniline raport FIPS PUB 197. NIST, 2001. <https://csrc.nist.gov/publications/detail/fips/197/final>.
- [16] Bitwarden Inc. Bitwarden Security Whitepaper. 2024. <https://bitwarden.com/help/bitwarden-security-white-paper/> (15.03.2025).
- [17] McGrew D. A. ja Viega J. The Security and Performance of the Galois/Counter Mode (GCM) of Operation. *Progress in Cryptology – INDOCRYPT 2004*. Springer, 2004, lk 343–355. DOI: [10.1007/978-3-540-30556-9_27](https://doi.org/10.1007/978-3-540-30556-9_27).
- [18] Krawczyk H. Cryptographic Extraction and Key Derivation: The HKDF Scheme. *Advances in Cryptology – CRYPTO 2010* (2010), lk 631–648. DOI: [10.1007/978-3-642-14623-7_34](https://doi.org/10.1007/978-3-642-14623-7_34).
- [19] Kaliski B. PKCS #5: Password-Based Cryptography Specification Version 2.0. Tehniline raport RFC 2898. IETF, 2000. <https://www.rfc-editor.org/rfc/rfc2898>.
- [20] Provos N. ja Mazières D. A Future-Adaptable Password Scheme. *USENIX Annual Technical Conference, FREENIX Track*. 1999, lk 81–91.
- [21] Percival C. Stronger Key Derivation via Sequential Memory-Hard Functions. *BSDCan 2009*. 2009. <https://www.tarsnap.com/scrypt/scrypt.pdf>.
- [22] Biryukov A., Dinu D. ja Khovratovich D. Argon2: New Generation of Memory-Hard Functions for Password Hashing and Other Applications. *2016 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2016, lk 292–302. DOI: [10.1109/EuroSP.2016.31](https://doi.org/10.1109/EuroSP.2016.31).
- [23] OWASP Foundation. Password Storage Cheat Sheet. 2024. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (15.03.2025).
- [24] Krawczyk H. ja Eronen P. HMAC-based Extract-and-Expand Key Derivation Function (HKDF). Tehniline raport RFC 5869. IETF, 2010. <https://www.rfc-editor.org/rfc/rfc5869>.
- [25] Goldwasser S., Micali S. ja Rackoff C. The Knowledge Complexity of Interactive Proof Systems. *SIAM Journal on Computing*. Köide 18. 1. 1989, lk 186–208. DOI: [10.1137/0218012](https://doi.org/10.1137/0218012).
- [26] OWASP Foundation. OWASP Top Ten. 2021. <https://owasp.org/www-project-top-ten/> (15.03.2025).

- [27] Cure53. Pentest-Report Bitwarden 2023. 2023. <https://bitwarden.com/resources/security-compliance/> (15.03.2025).
- [28] AgileBits Inc. 1Password Security Design. 2024. <https://1passwordstatic.com/files/security/1password-white-paper.pdf> (15.03.2025).
- [29] LastPass. LastPass Technical Whitepaper. 2023. [https://support.lastpass.com/s/document-item?language=en_US&bundleId=lastpass&topicId=LastPass/LastPass Technical Whitepaper.html](https://support.lastpass.com/s/document-item?language=en_US&bundleId=lastpass&topicId=LastPass/LastPass+Technical+Whitepaper.html) (15.03.2025).
- [30] Reichl D. KeePass Password Safe. 2024. <https://keepass.info/> (15.03.2025).
- [31] Li Z., He W., Akhawe D. ja Song D. The Emperor's New Password Manager: Security Analysis of Web-based Password Managers. *23rd USENIX Security Symposium*. 2014, lk 465–479.
- [32] Oesch S. ja Ruoti S. That Was Then, This Is Now: A Security Evaluation of Password Generation, Storage, and Autofill in Browser-Based Password Managers. *29th USENIX Security Symposium*. 2020, lk 2165–2182.
- [33] Stock B. ja Johns M. Protecting Users Against XSS-based Password Manager Abuse. *9th ACM Symposium on Information, Computer and Communications Security*. 2014, lk 183–194. DOI: [10.1145/2590296.2590336](https://doi.org/10.1145/2590296.2590336).
- [34] Wheeler D. L. zxcvbn: Low-Budget Password Strength Estimation. *25th USENIX Security Symposium*. 2016, lk 157–173.
- [35] Brooke J. SUS – A Quick and Dirty Usability Scale. *Usability Evaluation in Industry* 189.194 (1996), lk 4–7.

Lisad

I. Lähtekoodi repositoorium

Rakenduse lähtekood on kättesaadav GitHubi repositooriumis:

<https://github.com/kasutajanimi/paroolihaldur>

Repositoorium sisaldab:

- `client/` — kliendipoolne React/TypeScript rakendus.
- `server/` — serveripoolne Node.js/Express rakendus.
- `docker-compose.yml` — arenduskeskkonna seadistus.
- `README.md` — paigaldus- ja kasutusjuhend.

II. Paigaldusjuhend

Eeldused

Rakenduse käivitamiseks on vaja:

1. Node.js versioon 18 või uuem.
2. PostgreSQL versioon 15 või uuem (või Docker).
3. npm versioon 9 või uuem.
4. Docker ja Docker Compose (valikuline, arenduskeskkonna jaoks).

Käivitamine Dockeriga

```
# Kloonib repositooriumi
git clone https://github.com/kasutajanimi/paroolihaldur.git
cd paroolihaldur

# Kopeeri keskkonna muutujate näidisfail
cp .env.example .env

# Käivita kõik teenused
docker-compose up -d

# Rakendus on nüüd kättesaadav aadressil http://localhost:3000
```

Käivitamine ilma Dockerita

```
# Kloonib repositoorium
git clone https://github.com/kasutajanimi/paroolihaldur.git
cd paroolihaldur

# Seadista andmebaas
cp .env.example .env
# Muuda .env failis DATABASE_URL vastavalt oma PostgreSQL seadistusele

# Paigalda serveri sõltuvused ja käivita migratsioonid
cd server
npm install
npx prisma migrate deploy
npm run dev &

# Paigalda kliendi sõltuvused ja käivita
cd ../client
npm install
npm run dev
```

III. Keskkonnamuutujad

Muutuja	Kirjeldus	Vaikeväärtus
DATABASE_URL	PostgreSQL ühenduse URL	—
JWT_SECRET	JWT allkirjastamise salajane võti	—
PORT	Serveri port	4000
CLIENT_URL	Kliendi aadress (CORS)	http://localhost:3000
NODE_ENV	Keskkonna tüüp	development

NB! JWT_SECRET peab olema vähemalt 256-bitine juhuslik väärtus. Selle genereerimiseks võib kasutada käsku: `openssl rand -hex 32`.

IV. Testide käivitamine

```
# Kliendi ühiktestid
cd client && npm test

# Serveri ühiktestid
```

```
cd server && npm test

# Integratsioonitested (nõuab jooksvat PostgreSQL-i)
cd server && npm run test:integration

# Kõik testid koos kaetuse raportiga
npm run test:coverage
```

V. Ekraanipildid

Sia tuleb lisada ekraanipildid rakenduse peamistest vaadetest.

Litsents

Lihtlitsents lõputöö reprodutseerimiseks ja üldsusele kättesaadavaks tegemiseks

Mina, Uku Renek Kronbergs,

1. annan Tartu Ülikoolile tasuta loa (lihtlitsentsi) minu loodud teose
Turvalise paroolihalduri veebirakenduse arendamine,
mille juhendaja on **Alo Peets**,
reprodutseerimiseks eesmärgiga seda säilitada, sealhulgas lisada digitaalarhiivi DSpace kuni autoriõiguse kehtivuse lõppemiseni.
2. Annan Tartu Ülikoolile loa teha punktis 1 nimetatud teos üldsusele kättesaadavaks Tartu Ülikooli veebikeskkonna, sealhulgas digitaalarhiivi DSpace kaudu Creative Commons'i litsentsiga CC BY NC ND 3.0, mis lubab autorile viidates teost reprodutseerida, levitada ja üldsusele suunata ning keelab luua tuletatud teost ja kasutada teost ärieesmärgil, kuni autoriõiguse kehtivuse lõppemiseni.
3. Olen teadlik, et punktides 1 ja 2 nimetatud õigused jäävad alles ka autorile.
4. Kinnitan, et lihtlitsentsi andmisega ei riku ma teiste isikute intellektuaalomandi ega isikuandmete kaitse õigusaktidest tulenevaid õigusi.

Uku Renek Kronbergs

23.02.2026